

Parallel Profiling and Applications

Lecture 26

April 20, 2026

NAS Parallel Benchmarks

<https://www.nas.nasa.gov/publications/npb.html>

- Five kernels
 - IS - Integer Sort, random memory access
 - EP - Embarrassingly Parallel
 - CG - Conjugate Gradient, irregular memory access and communication
 - MG - Multi-Grid on a sequence of meshes, long- and short-distance communication, memory intensive
 - FT - discrete 3D fast Fourier Transform, all-to-all communication
- Three pseudo applications
 - BT - Block Tri-diagonal solver
 - SP - Scalar Penta-diagonal solver
 - LU - Lower-Upper Gauss-Seidel solver

Aggregate Time – Strong Scaling Comparison (mg)

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	COV
Bcast	12	22.5	0.49	19.71	0.66
Send	9	21.1	0.46	18.47	0.10
Send	20	14.1	0.31	12.39	0.02
Send	1	13.4	0.29	11.74	0.32

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	COV
Barrier	7	149	2.24	21.84	0.01
Send	9	140	2.10	20.48	0.81
Send	21	123	1.84	17.94	0.87
Wait	26	58.8	0.88	8.60	0.09

Processes = 4

Processes = 16

Aggregate Time – Strong Scaling Comparison (ft)

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	COV
Alltoall	9	443	5.69	84.89	0.03
Bcast	7	43.3	0.56	8.29	0.00
Reduce	4	32.4	0.42	6.21	0.02
Barrier	5	1.57	0.02	0.30	1.16

Processes = 4

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	COV
Alltoall	9	1.73e+03	16.22	91.43	0.05
Reduce	4	76.4	0.72	4.03	0.84
Comm_split	10	44.3	0.41	2.34	0.92
Bcast	7	24.8	0.23	1.31	0.48

Processes = 16

Aggregate Time – Data Scaling (cg on 16 processes)

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	COV
Bcast	3	477	16.58	39.24	0.01
Wait	21	176	6.11	14.46	0.74
Send	10	162	5.64	13.34	0.85
Wait	6	89.6	3.11	7.37	0.13

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	COV
Wait	21	1.03e+04	3.07	31.48	0.79
Send	15	8.84e+03	2.65	27.13	0.19
Send	12	8.44e+03	2.53	25.91	0.79
Wait	11	728	0.22	2.23	1.49

Class = A
(small problem)

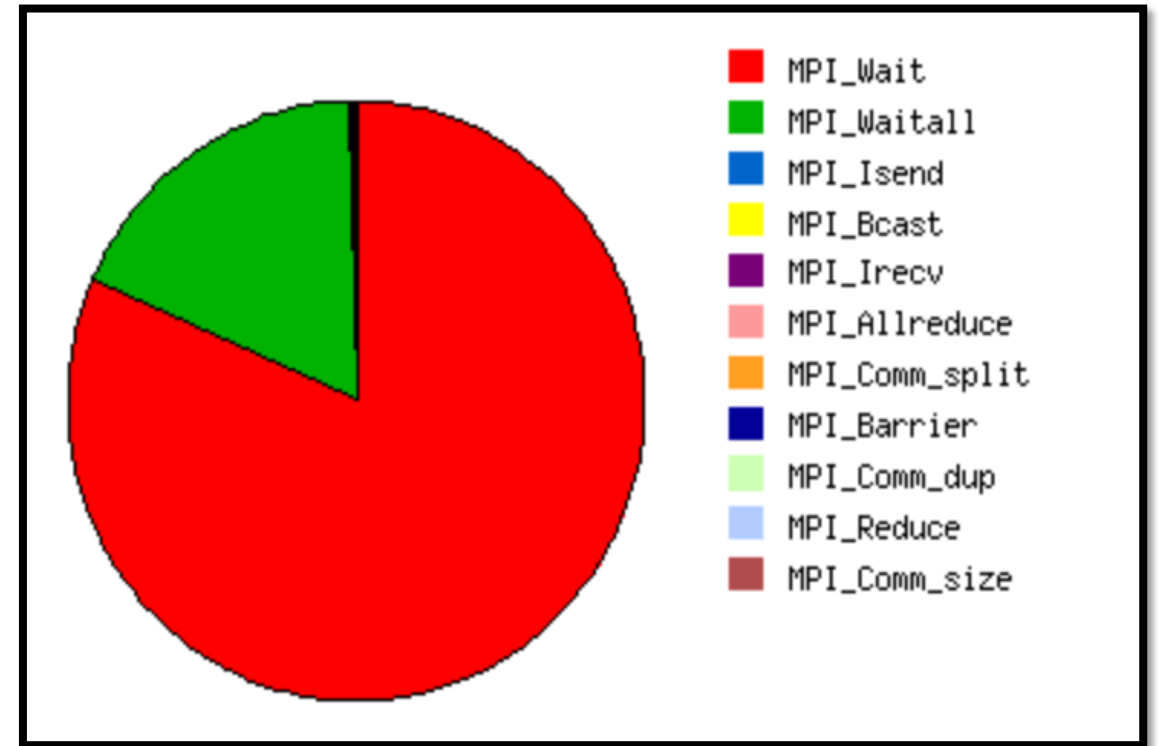
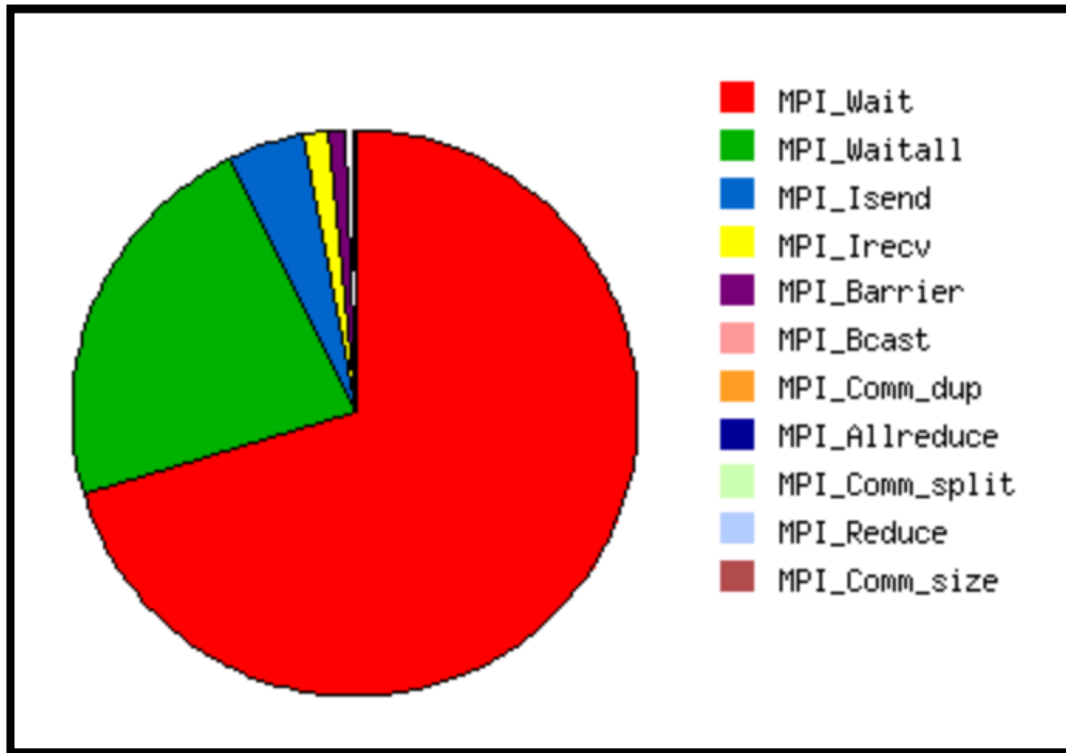
Class = C
(large problem)

MPI Time vs. App Time (Class = A NPROCS=16)

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	COV	
Bcast	3	477	16.58	39.24	0.01	cg
Allreduce	5	246	1.31	77.45	0.60	ep
Alltoall	9	1.73e+03	16.22	91.43	0.05	ft
Recv	3	3.78e+03	5.30	27.16	0.76	lu
Barrier	7	149	2.24	21.84	0.01	mg
Waitall	41	1.89e+03	2.13	20.52	0.17	sp

IPM Profile Comparison



What may be the **runtime** configuration differences?

Timings

```
class$ mpirun -np 2 ./send 1000  
0 1000 0.000020  
1 1000 0.000024
```

```
class$ mpirun -np 2 ./send 1000  
0 1000 0.000026  
1 1000 0.000113
```

```
class$ mpirun -np 2 ./send 10000000  
0 10000000 0.010407  
1 10000000 0.010231
```

```
class$ mpirun -np 2 ./send 100000000  
0 100000000 0.072819  
1 100000000 0.070379
```

```
class$ mpirun -np 2 ./send 1000  
0 1000 0.000020  
1 1000 0.000032
```

```
class$ mpirun -np 2 ./send 1000  
0 1000 0.000019  
1 1000 0.000021
```

```
class$ mpirun -np 2 ./send 10000000  
0 10000000 0.010318  
1 10000000 0.010271
```

```
class$ mpirun -np 2 ./send 100000000  
0 100000000 0.075876  
1 100000000 0.073563
```


Timings

```
class$ mpirun -np 2 ./send 1000000000  
0 1000000000 0.072819  
1 1000000000 0.070379
```

```
0 1000000000 0.044909  
1 1000000000 0.040942
```

Same run command,
why is the time less?

```
class$ mpirun -np 2 ./send 1000000000  
0 1000000000 0.075876  
1 1000000000 0.073563
```

```
0 1000000000 0.044516  
1 1000000000 0.040513
```

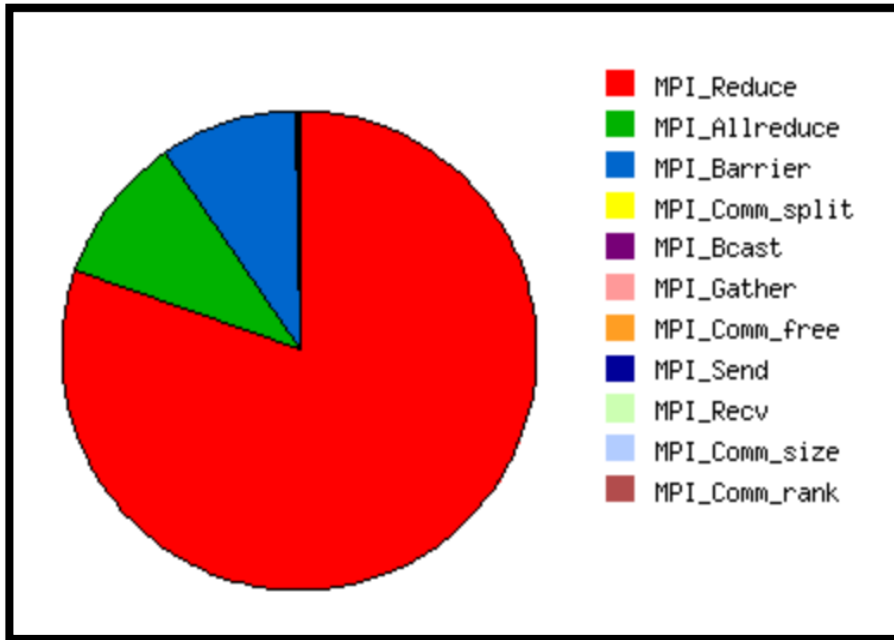
Intel MPI Benchmarks (IMB)

<https://github.com/intel/mpi-benchmarks>

- Bcast/multi-Bcast
- Allgather/multi-Allgather
- Allgatherv/multi-Allgatherv
- Alltoall/multi-Alltoall
- Alltoallv/multi-Alltoallv
- Scatter/multi-Scatter
- Scatterv/multi-Scatterv
- Gather/multi-Gather
- Gatherv/multi-Gatherv
- Reduce/multi-Reduce
- Reduce_scatter/multi-Reduce_scatter
- Allreduce/multi-Allreduce
- Barrier/multi-Barrier

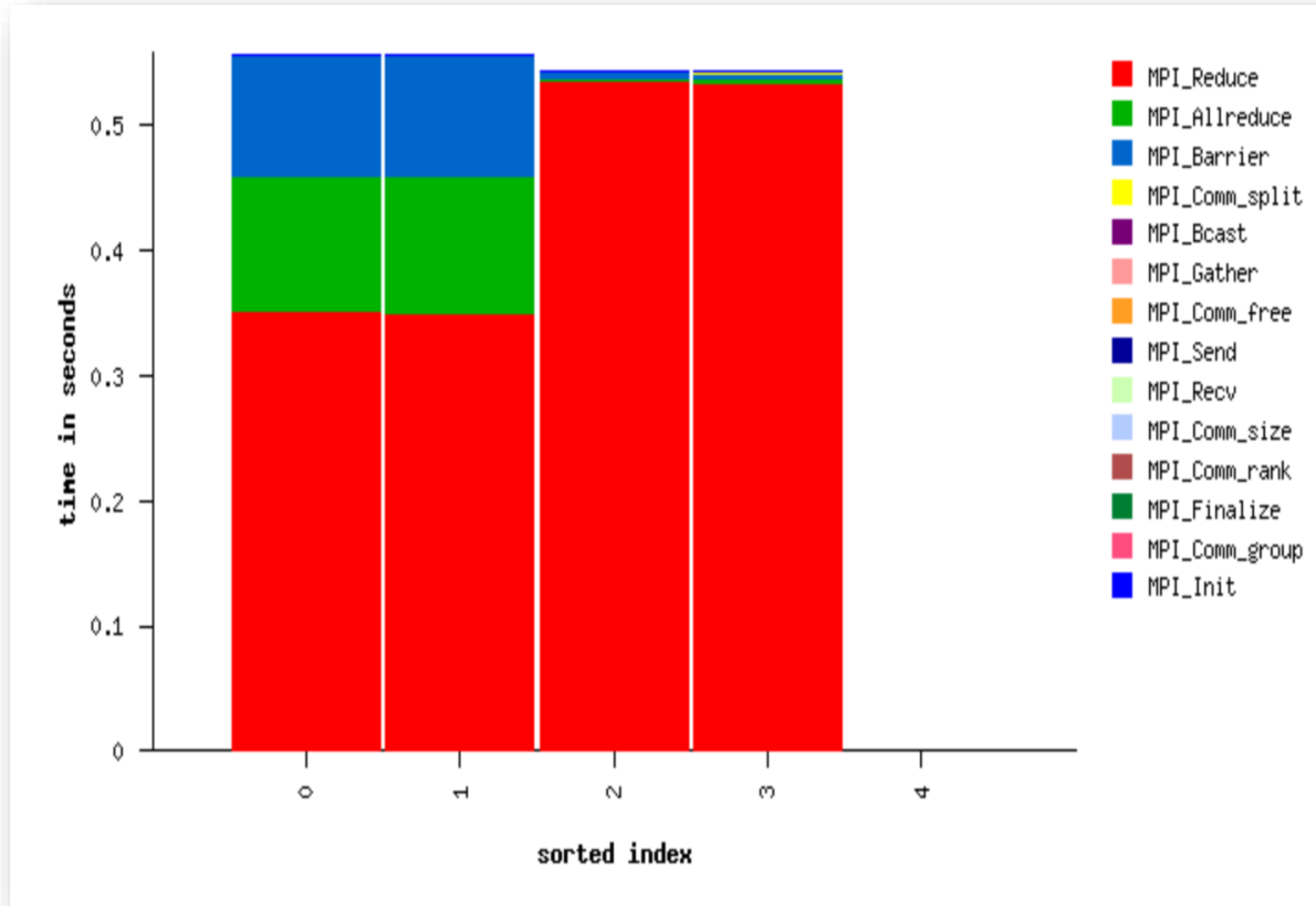
```
For (msg_size = small to large)
{
    MPI_Barrier();
    For (some number of iterations)
    {
        MPI_XYZ (... msg_size ...);
    }
}
```

IMB Reduce (NPROCS = 4)

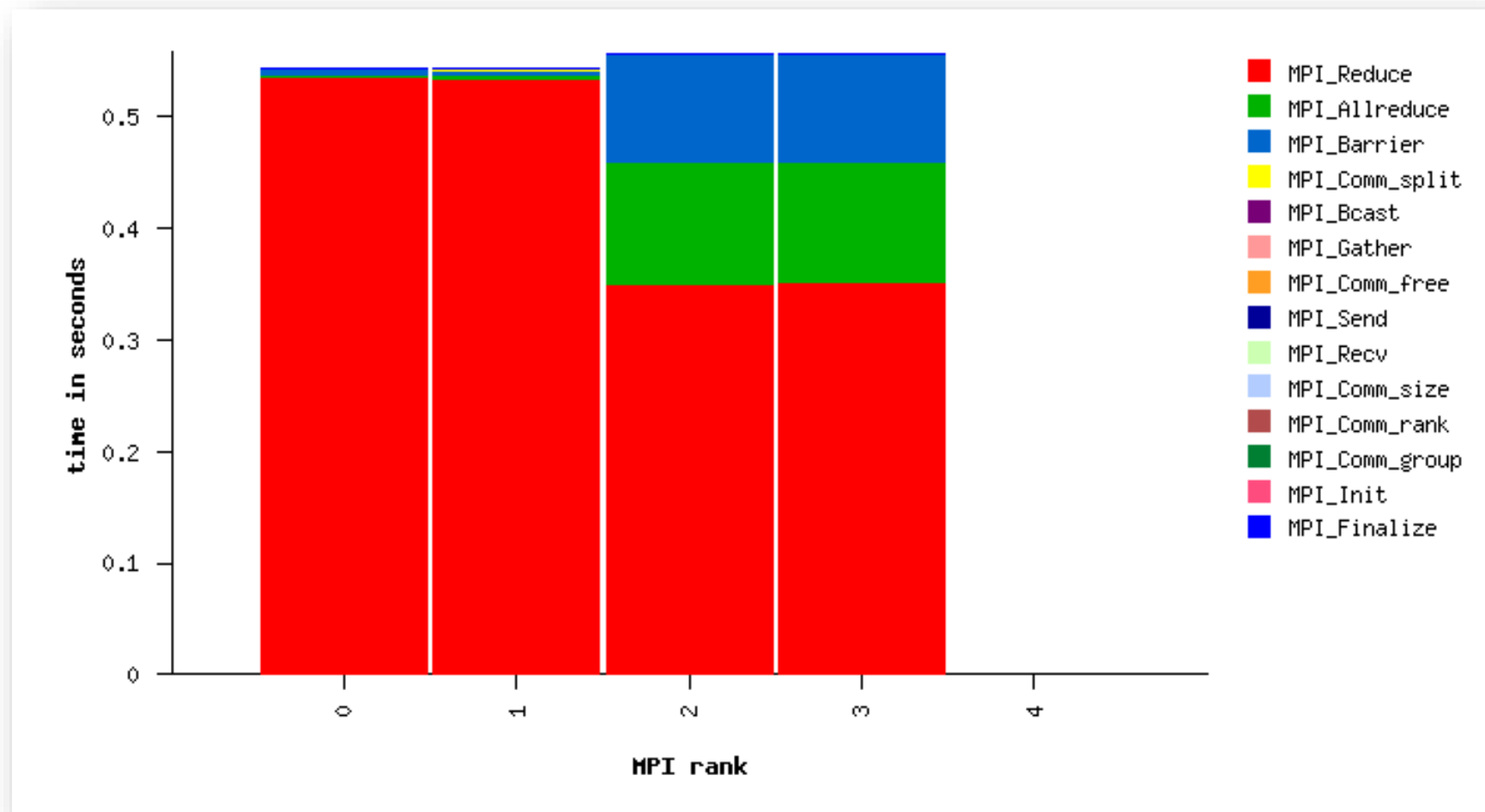


Communication Event Statistics (100.00% detail, 7.4219e-07 error)			
	Buffer Size	Ncalls	Total Time
MPI_Reduce	32768	12006	0.222
MPI_Allreduce	4	308	0.217
MPI_Barrier	0	984	0.205
MPI_Reduce	4194304	126	0.203
MPI_Reduce	2097152	246	0.181
MPI_Reduce	65536	7686	0.167
MPI_Reduce	131072	3846	0.154
MPI_Reduce	1048576	486	0.147
MPI_Reduce	262144	1926	0.145
MPI_Reduce	524288	966	0.141
MPI_Reduce	16384	12006	0.121
MPI_Reduce	8192	12006	0.065
MPI_Reduce	4096	12006	0.039
MPI_Reduce	2048	12006	0.028
MPI_Reduce	1024	12006	0.023
MPI_Reduce	512	12006	0.019

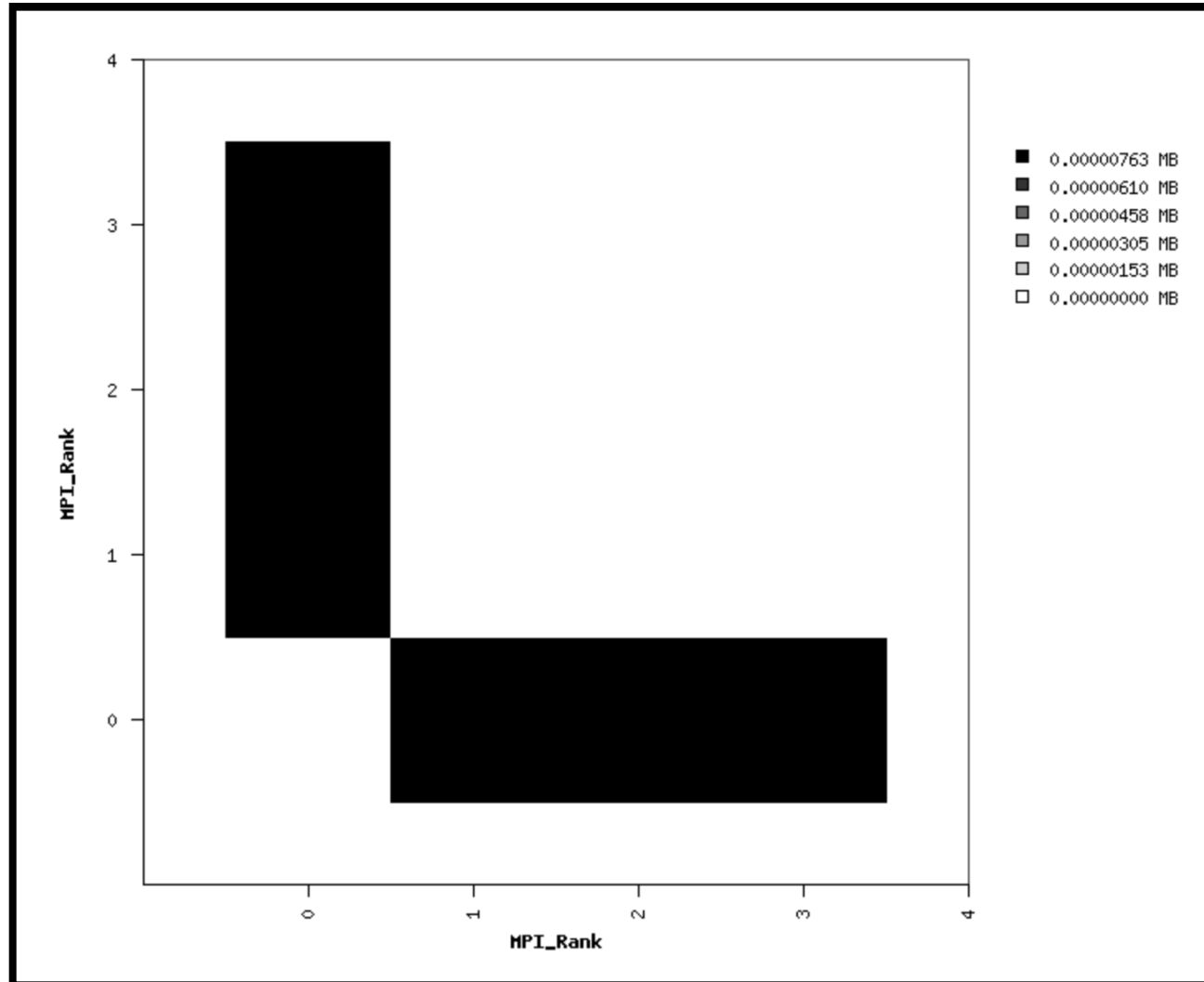
IMB Reduce (NPROCS = 4, default: sorted by total time)



Sorted by Rank



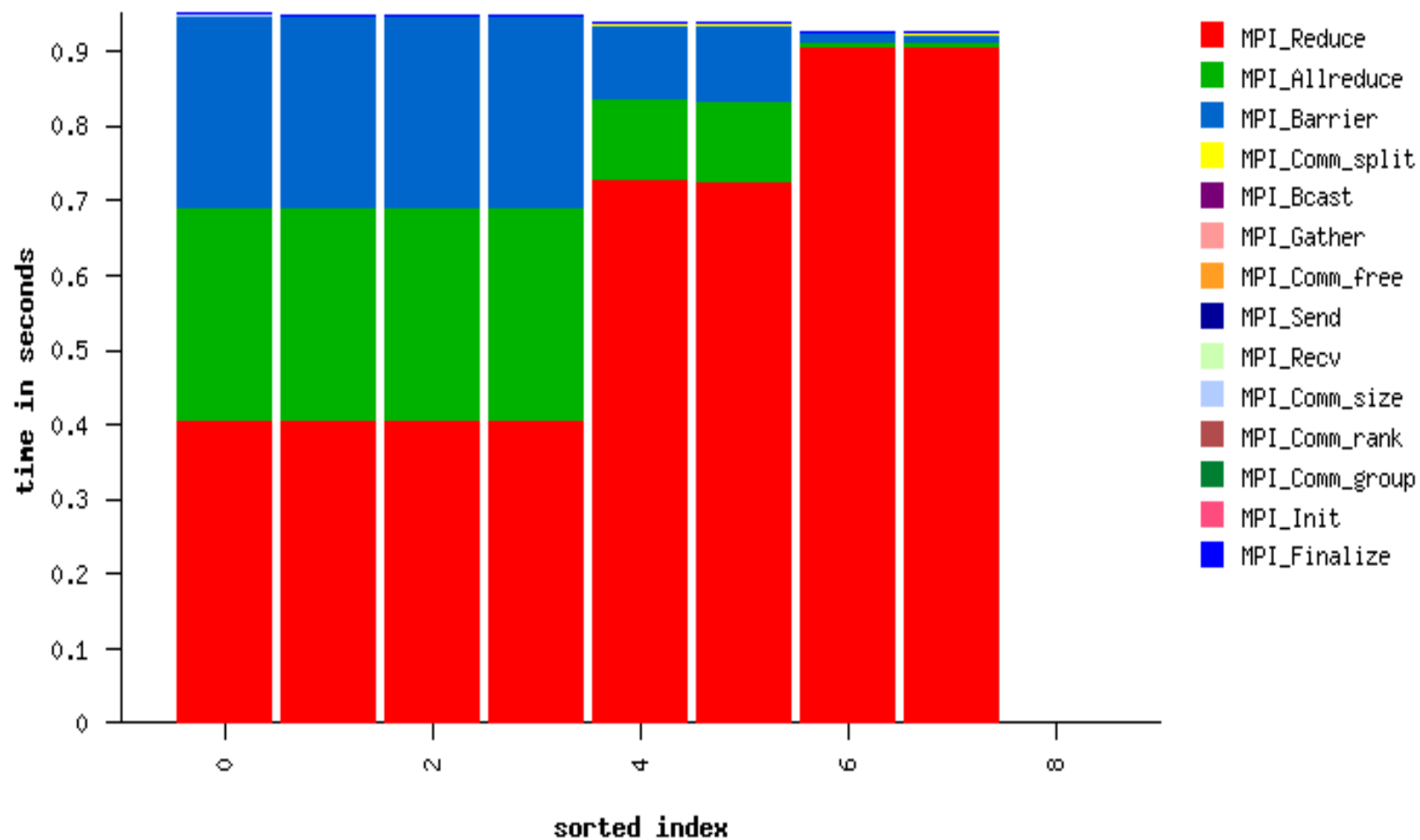
Communication Matrix (IMB Reduce, NPROCS=4)



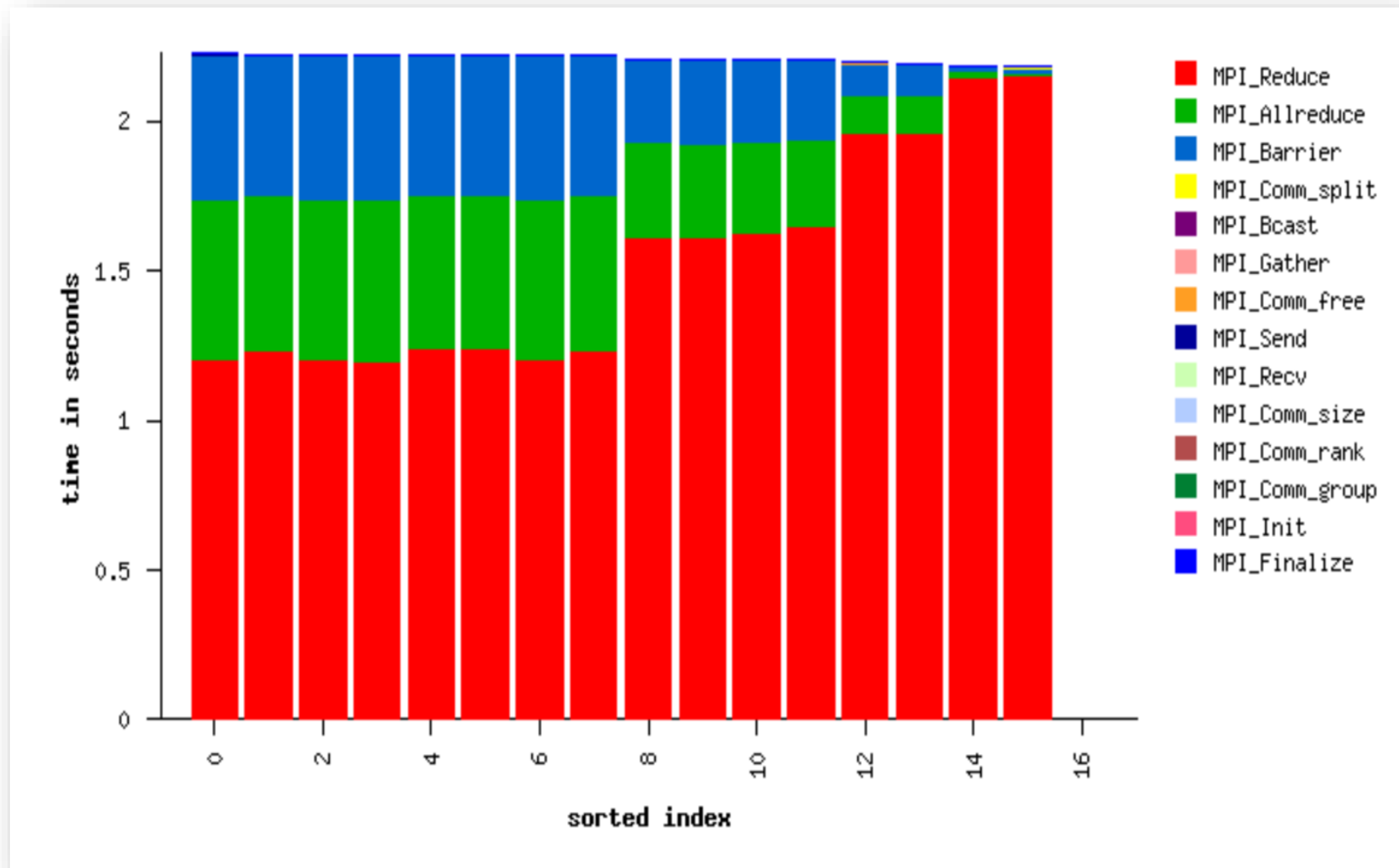
This communication pattern is due to the P2P during initialization (note the small data size)

The collective step by step P2P are not shown in almost any of the widely-used profilers

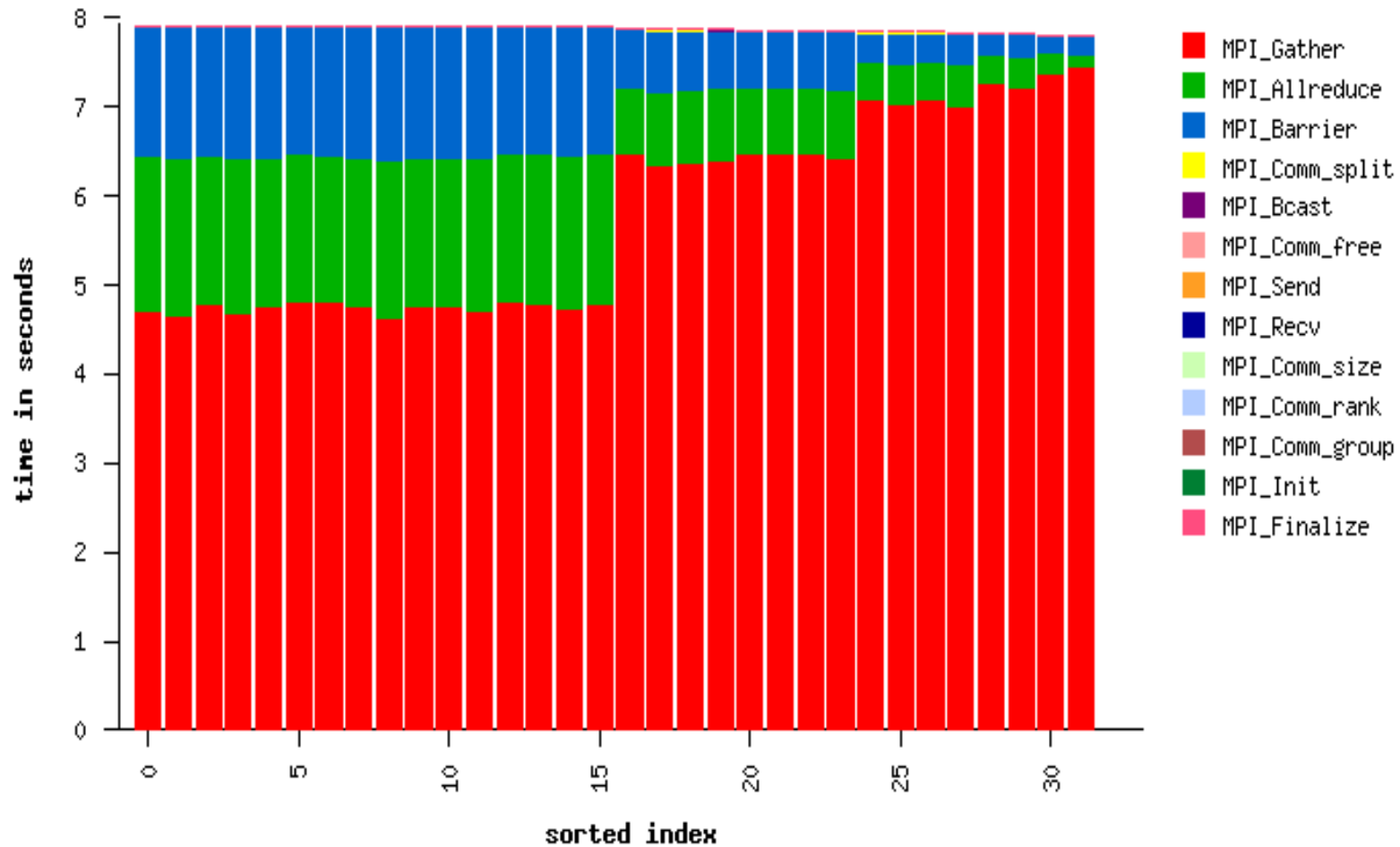
IMB Reduce (NPROCS = 8, 1 host)



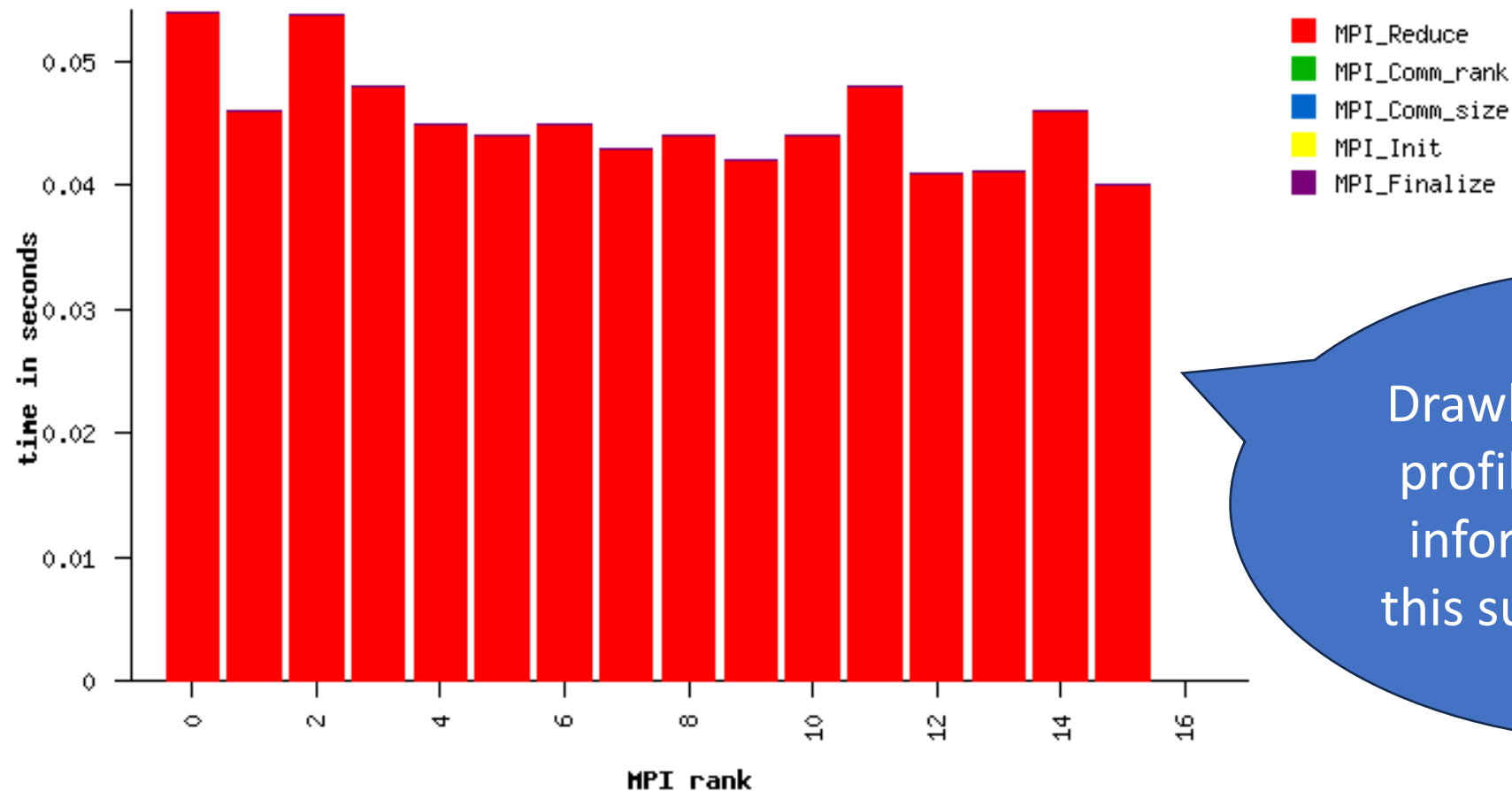
IMB Reduce (NPROCS = 16, 2 hosts)



IMB Gather (NPROCS=32, 4 hosts)

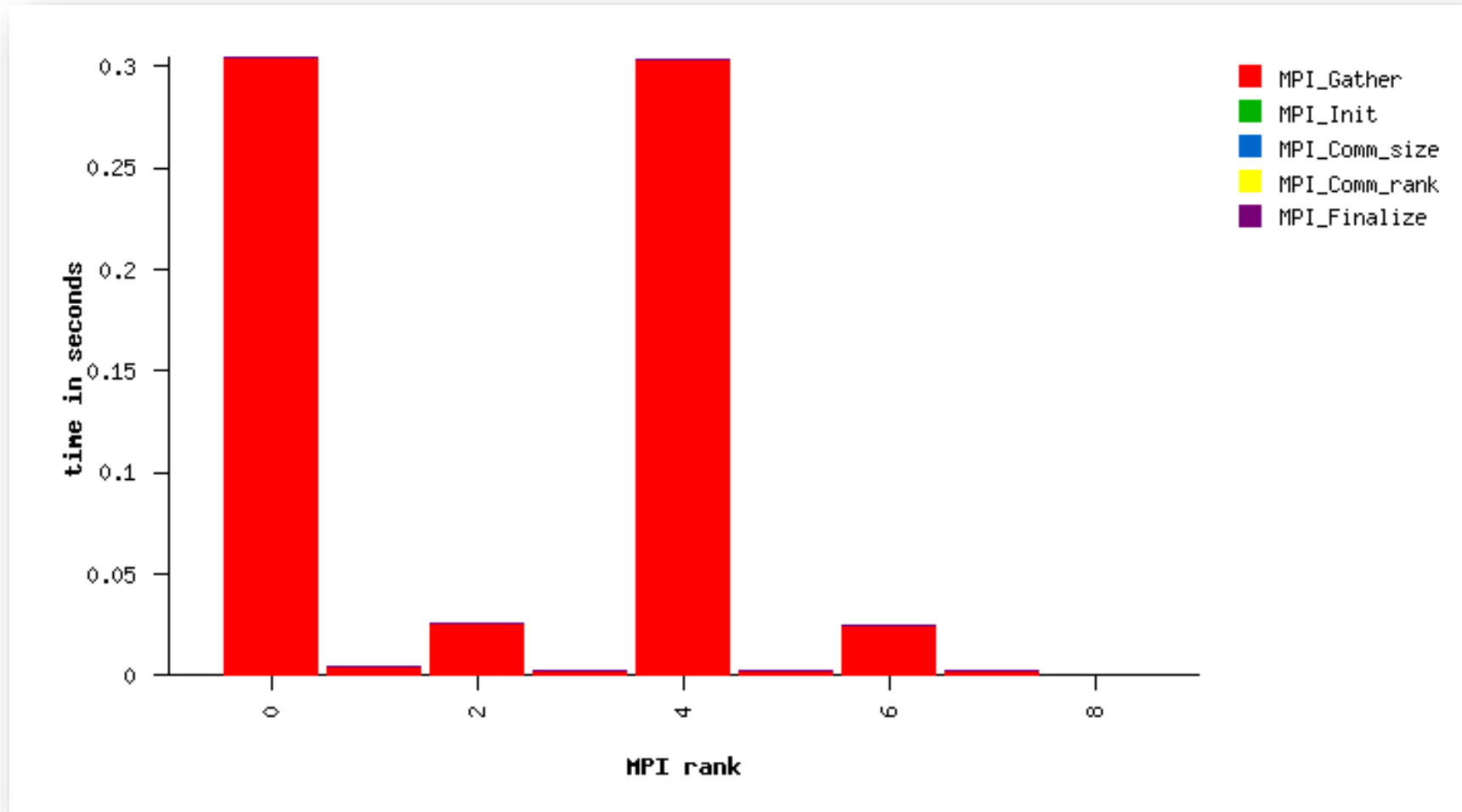


MPI_Reduce (sorted by rank, multiple calls, different message sizes, different root ranks)

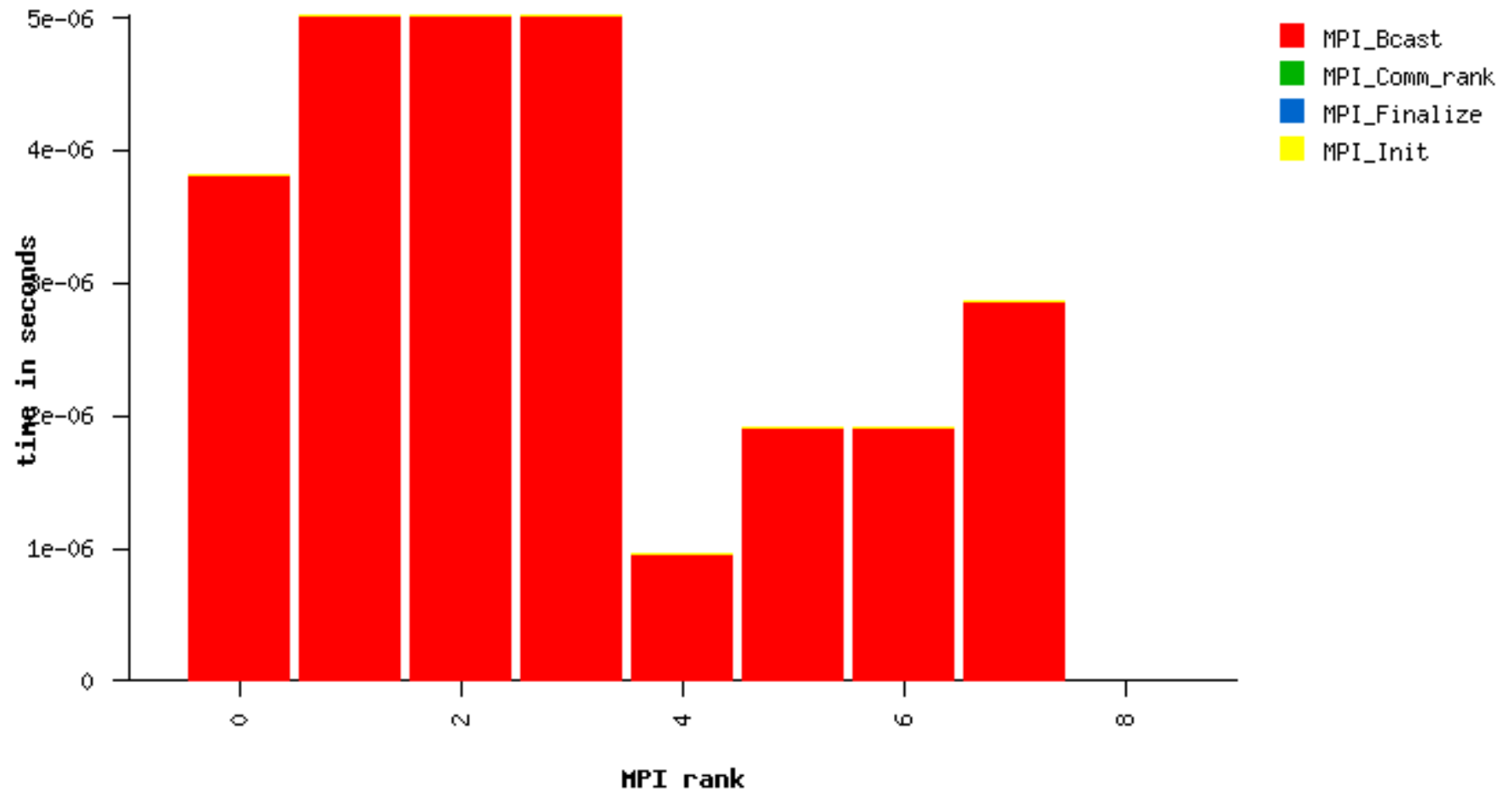


Drawback of summary profile, individual call information is lost in this summary diagram.

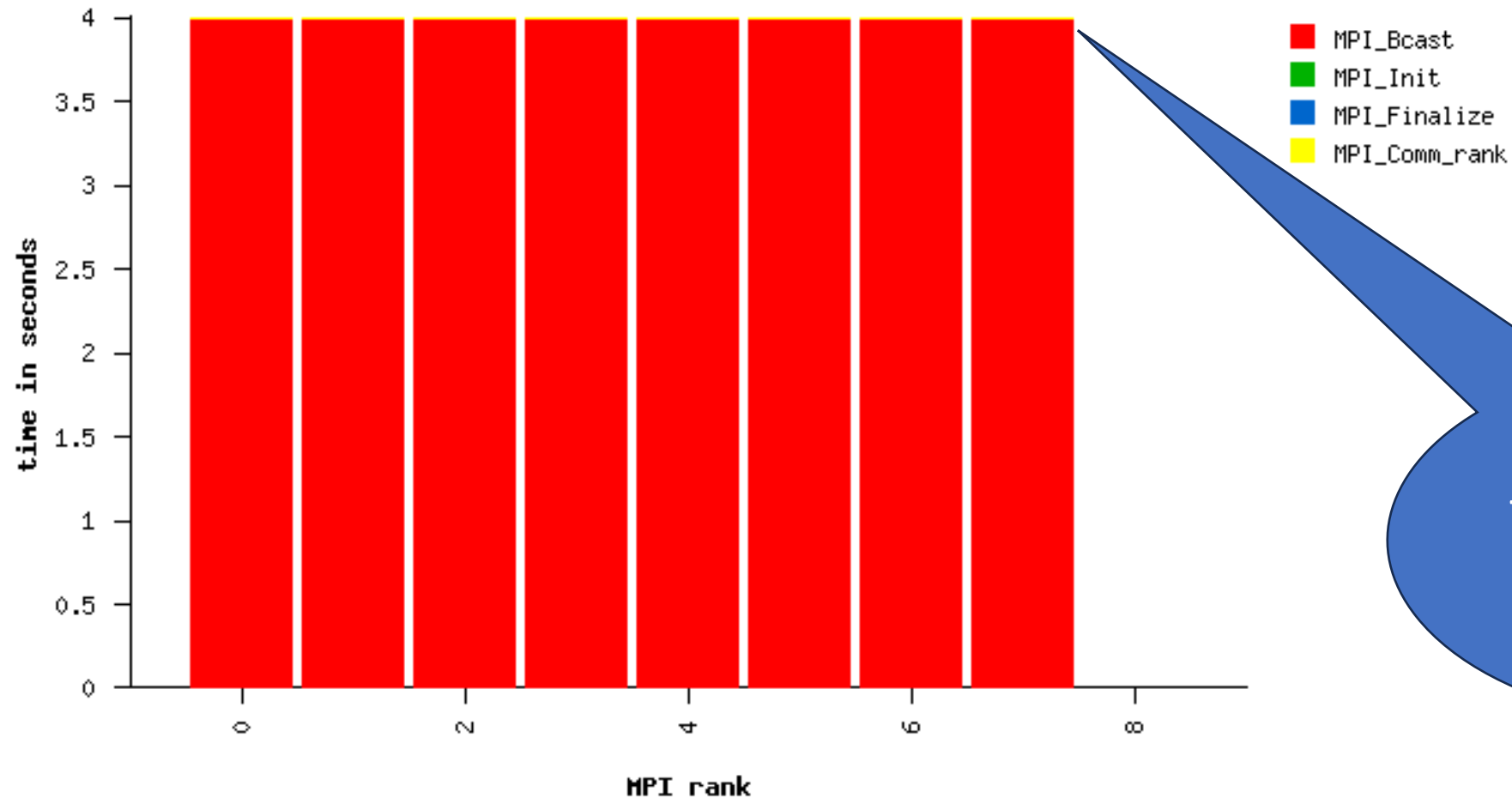
MPI_Gather (sorted by rank, one call)



MPI_Bcast (One iteration, root = 0)



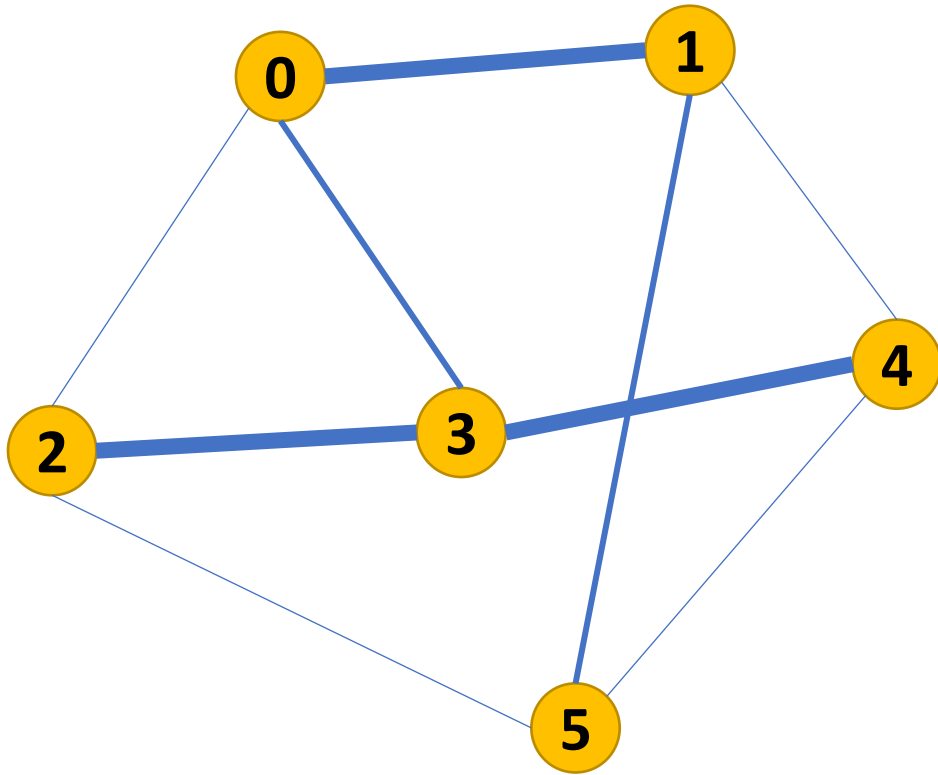
MPI_Bcast (Several iterations)



Variability is still there, but it is less visible due to the scale (y-axis)

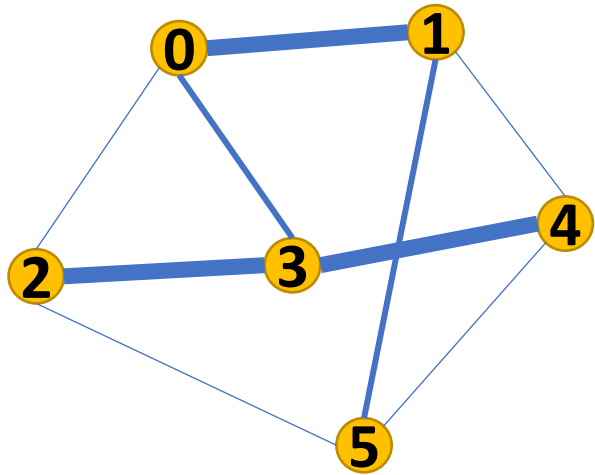
Communication Graph/Matrix

64, 256, 512 bytes (lighter to darker shade)



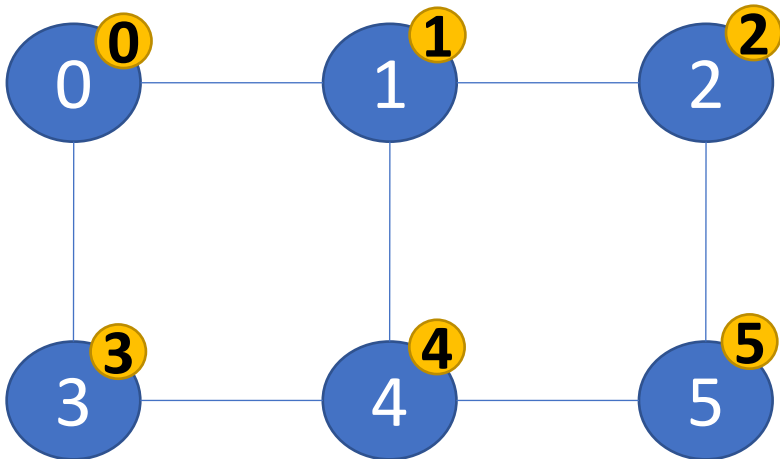
	512	64	256		
512				64	256
64			512		64
256		512		512	
	64		512		64
	256	64		64	

Communication Graph Mapping



	512		256		
512				64	256
			512		64
256		512		512	
	64		512		64
	256	64		64	

Linear/sequential mapping



Q1: What are the communicating pairs?

Q2: Distance/hops between the communicating pairs?

Q3: Total hop-bytes?

Process Mapping

- Processor topology graph
- Task graph (process)
 - $c_{ab} \rightarrow$ amount of communication (bytes) on e_{ab} between v_a and v_b
- Map: $v_t \in V_t$ placed on $v_p \in V_p$

$$G_p = (V_p, E_p)$$

$$G_t = (V_t, E_t)$$

$$P : V_t \longrightarrow V_p$$

Metric 1: Hop-bytes

Total size of inter-processor communication in bytes weighted by distance between the respective end-processors.

The overall hop-bytes is the sum of hop-bytes due to individual nodes in the task graph.

$$HB(G_t, G_p, P) = \frac{1}{2} \sum_{v_a \in V_t} HB(v_a)$$

$$\text{where } HB(v_a) = \sum_{e_{ab} \in E_t} HB(e_{ab})$$

$$\text{where } HB(e_{ab}) = c_{ab} \times d_p(P(v_a), P(v_b))$$

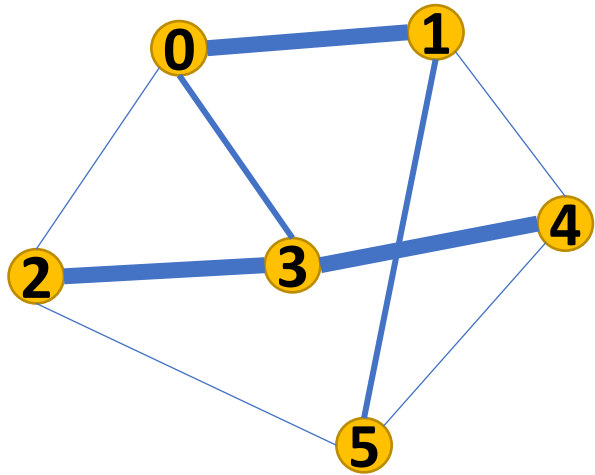
Metric 2: Hops per byte

Average number of network links a byte has to travel under a task mapping

$$\text{Hops per Byte} = \frac{HB(G_t, G_p, P)}{\sum_{e_{ab} \in E_t} c_{ab}}$$

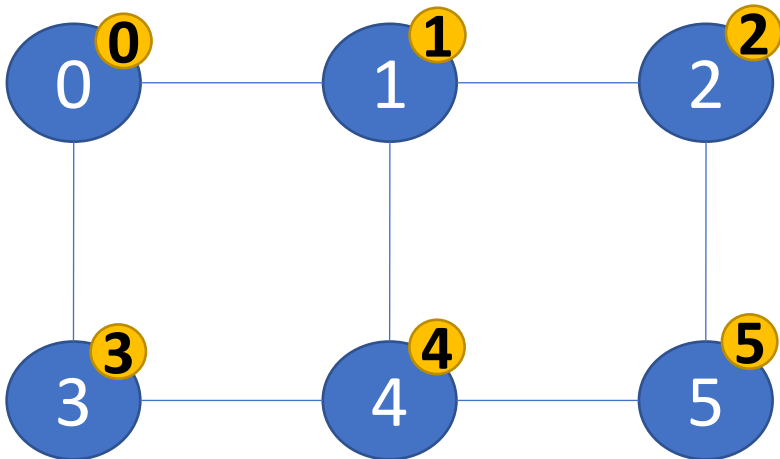
$$\text{Hops per Byte} = \frac{\sum_{e_{ab} \in E_t} c_{ab} \times d_p(P(v_a), P(v_b))}{\sum_{e_{ab} \in E_t} c_{ab}}$$

Communication Graph Mapping



	512	64	256		
512				64	256
64			512		64
256		512		512	
	64		512		64
	256	64		64	

Linear mapping



Total hop-bytes?

01: $512 * 1$

02: $64 * 2$

03: $256 * 1$

14: $64 * 1$

15: $256 * 2$

23: $512 * 3$

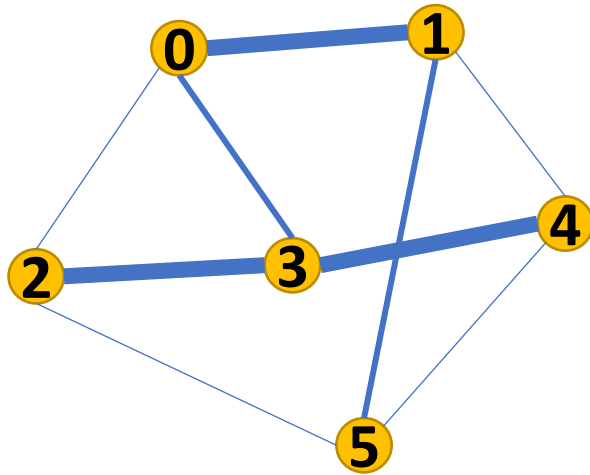
25: $64 * 1$

34: $512 * 1$

45: $64 * 1$

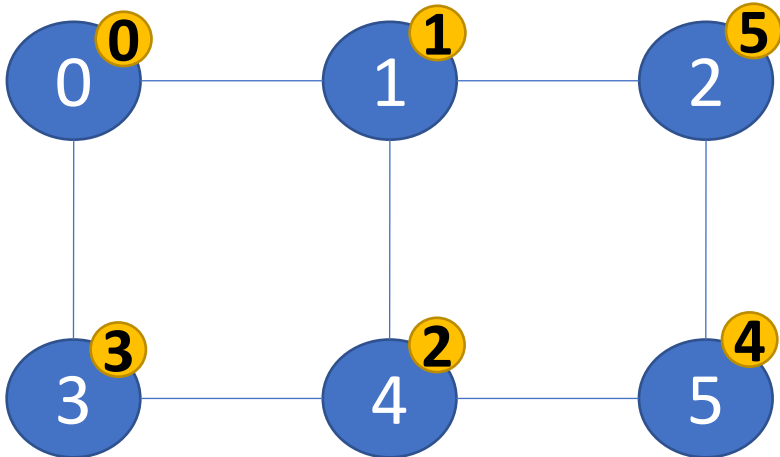
= 3648

Communication Graph Mapping



	512	64	256		
512				64	256
64			512		64
256		512		512	
	64		512		64
	256	64		64	

A different mapping (to reduce the hop-bytes)



Total hop-bytes?

01: 512×1

02: 64×2

03: 256×1

14: 64×2

15: 256×1

23: 512×1

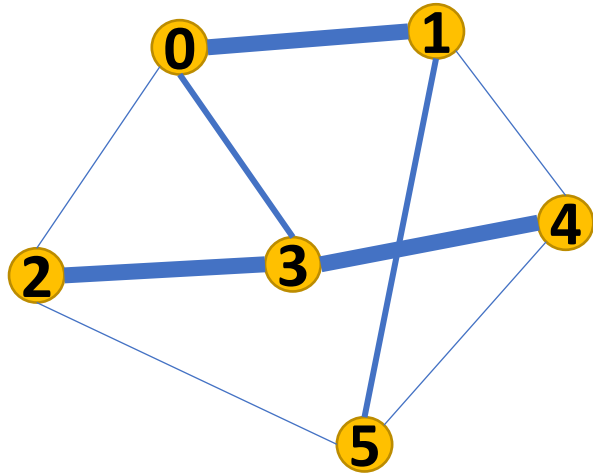
25: 64×2

34: 512×2

45: 64×1

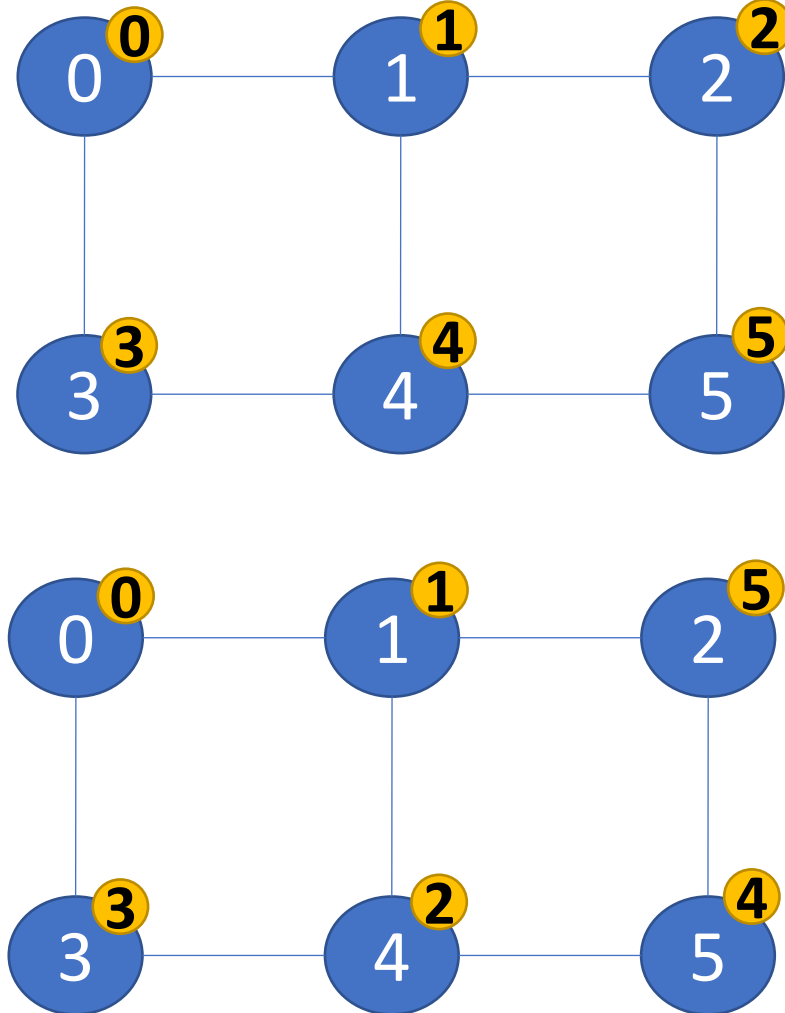
= 3008

Communication Graph Mapping



	512	64	256		
512				64	256
64			512		64
256		512		512	
	64		512		64
	256	64		64	

Linear mapping



Linear

01: 512*1
02: 64*2
03: 256*1
14: 64*1
15: 256*2
23: 512*3
25: 64*1
34: 512*1
45: 64*1

= 3648

Towards optimal

01: 512*1
02: 64*2
03: 256*1
14: 64*2
15: 256*1
23: 512*1
25: 64*2
34: 512*2
45: 64*1

= 3008

Effect of Mapping

- 16 x 16 x 16 3D torus
 - Diameter: 24
- Average load on links high when #hops large and for large message size

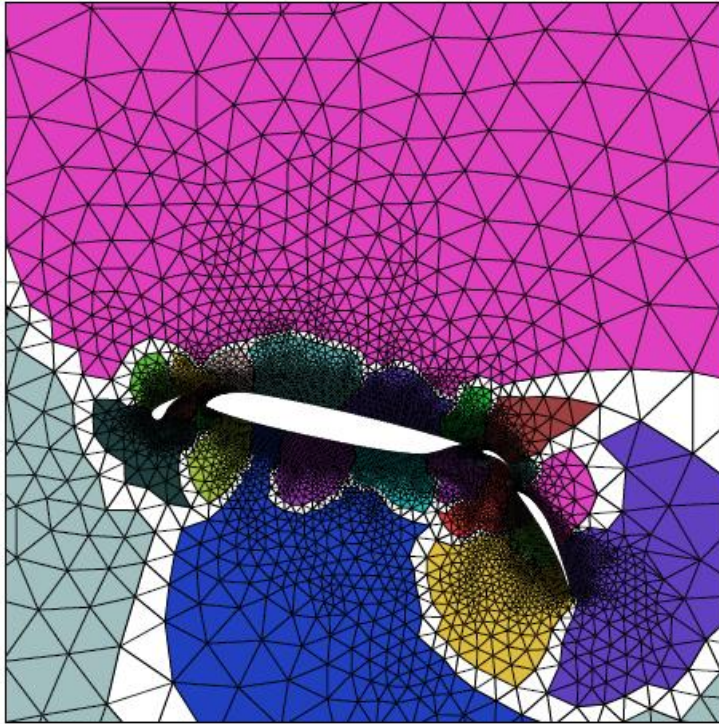
Message Size	Random Mapping	Optimal Mapping
1KB	56.93ms	46.91ms
10KB	243.64ms	124.56ms
100KB	2247.75ms	914.72ms
500KB	11.62s	4.44s
1MB	23.50s	8.80s

17%

62%

Table 1: Time for 200 iterations of a Jacobi-like program with optimal mapping and random mapping

Irregular Mesh

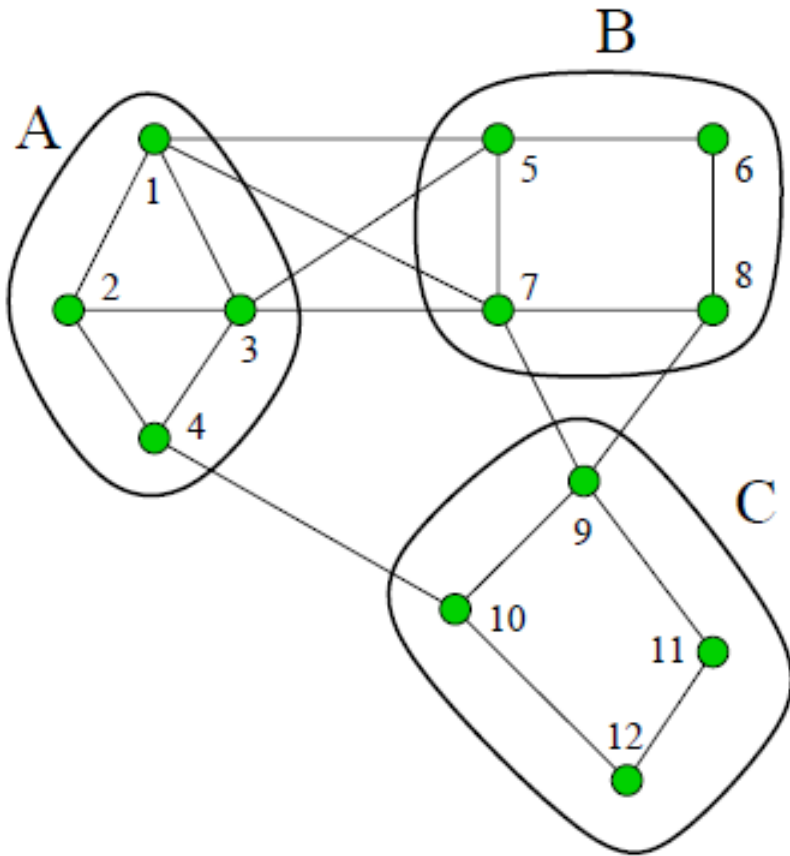


2D irregular mesh of an airfoil *

Q: How can you ensure efficient execution of these simulations?

Q: How do you model irregular mesh computations/communications as graphs?

Graph Partitioning



Three subdomains/parts: A, B, C
(assuming we have 3 nodes)

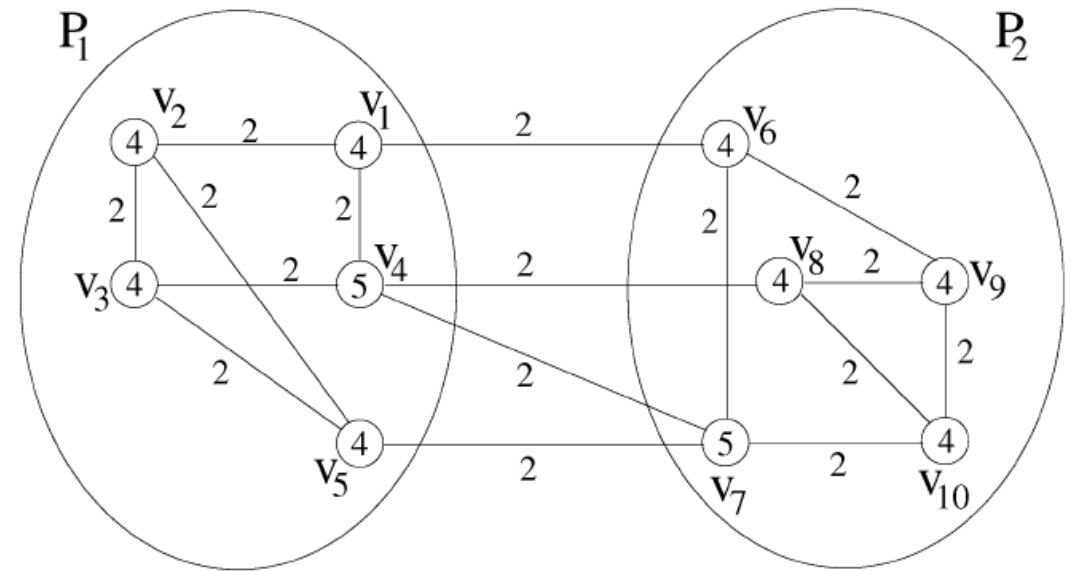
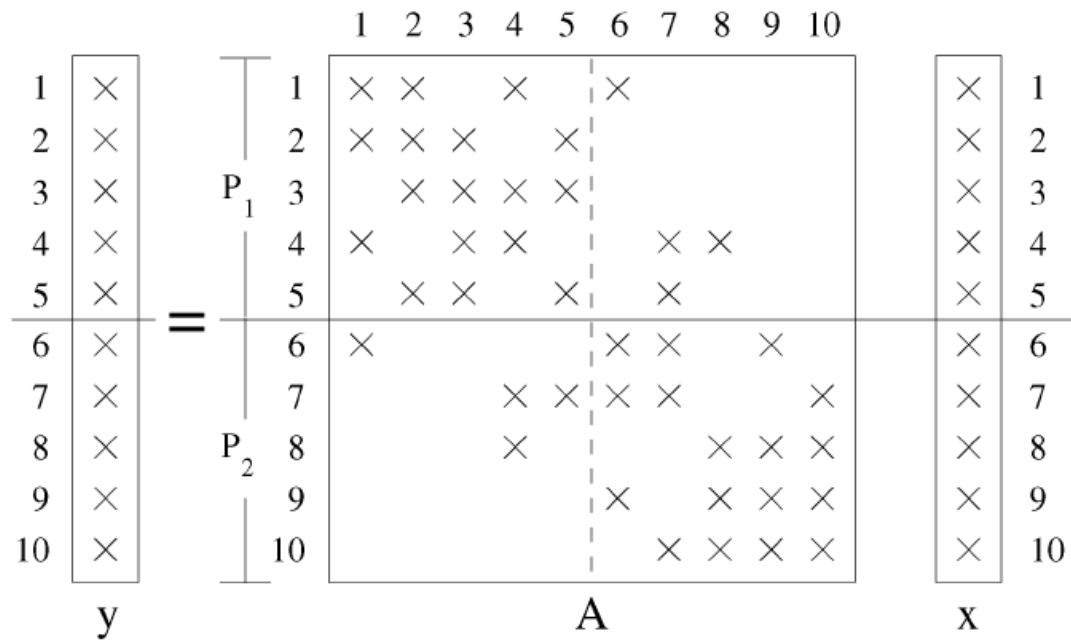
Edge is a *cut* when its vertex pairs belong to two parts

Edge cut = 7

Communication cost $\propto$ edge-cut

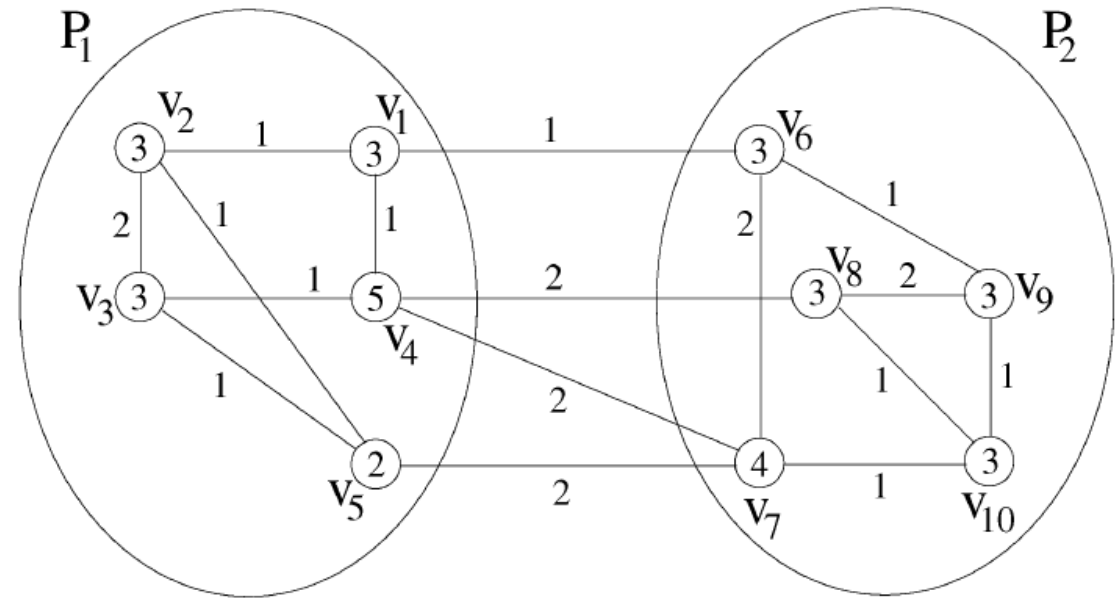
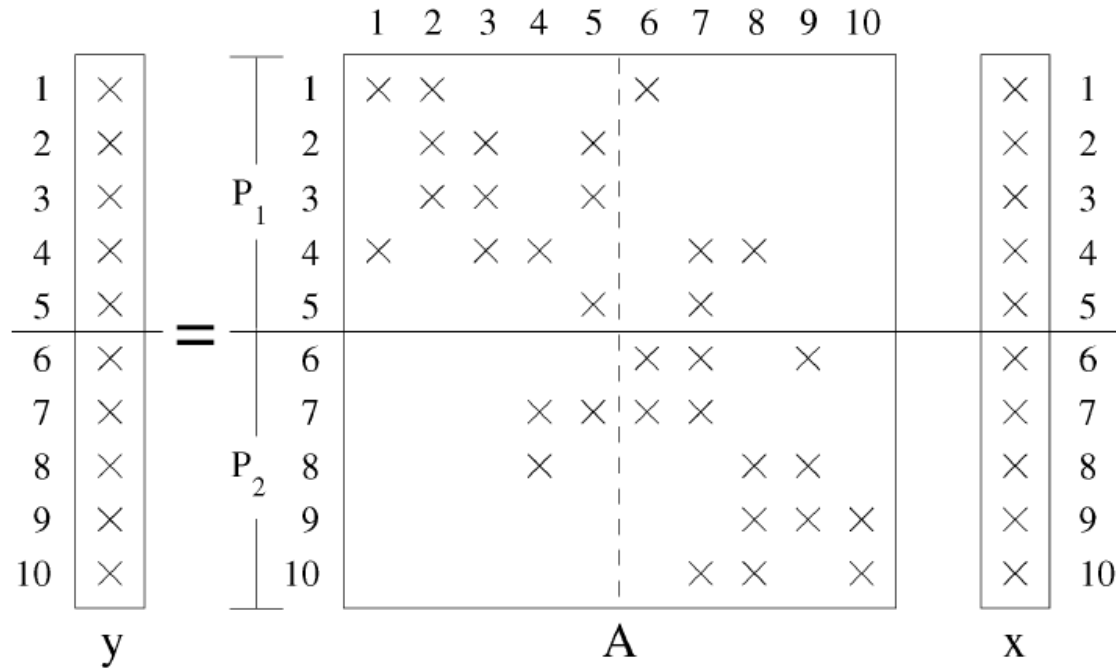
*Hypergraph-Partitioning-Based Decomposition for Parallel Sparse-Matrix Vector Multiplication

Graph Model for SpMV – Symmetric Matrix



(Row-wise) Two-way partitioning of symmetric matrix for SpMV computation
 Computational load w_i = Number of non-zero entries in row/column i

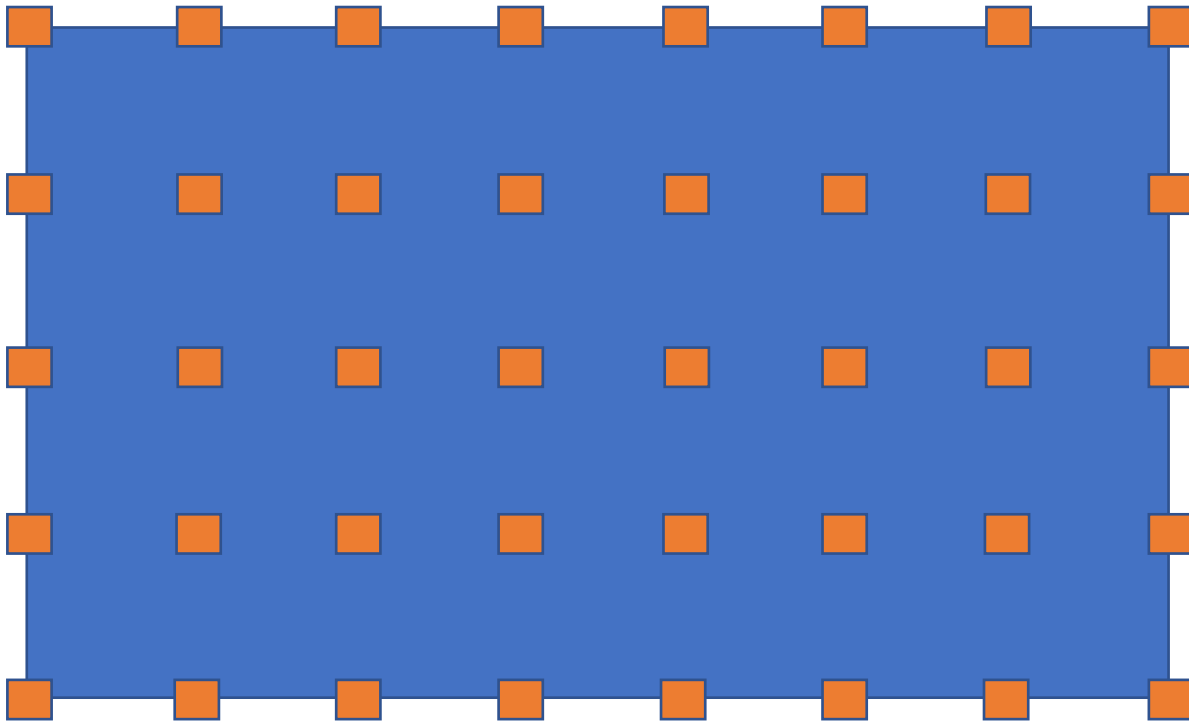
Communication Graph – Non-symmetric Matrix



Examples

matrix name	description	number of rows/cols	number of nonzeros									
			total	avg. per row/col	per column				per row			
					min	max	std	cov	min	max	std	cov
Structurally Symmetric Matrices												
SHERMAN3	[9] 3D finite difference grid	5005	20033	4.00	1	7	2.66	0.67	1	7	2.66	0.67
KEN-11	[7] linear programming	14694	82454	5.61	2	243	14.54	2.59	2	243	14.54	2.59
NL	[19] linear programming	7039	105089	14.93	1	361	28.48	1.91	1	361	28.48	1.91
KEN-13	[7] linear programming	28632	161804	5.65	2	339	16.84	2.98	2	339	16.84	2.98
CQ9	[19] linear programming	9278	221590	23.88	1	702	54.46	2.28	1	702	54.46	2.28
CO9	[19] linear programming	10789	249205	23.10	1	707	52.17	2.26	1	707	52.17	2.26
CRE-D	[7] linear programming	8926	372266	41.71	1	845	76.46	1.83	1	845	76.46	1.83
CRE-B	[7] linear programming	9648	398806	41.34	1	904	74.69	1.81	1	904	74.69	1.81
FINAN512	[38] stochastic programming	74752	615774	8.24	3	1449	20.00	2.43	3	1449	20.00	2.43
Structurally Nonsymmetric Matrices												
GEMAT11	[9] optimal power flow	4929	38101	7.73	1	28	2.96	0.38	1	29	3.38	0.44
LHR07	[38] light hydrocarbon recovery	7337	163716	22.31	1	64	26.19	1.17	2	37	16.00	0.72
ONETONE2	[38] nonlinear analog circuit	36057	254595	7.06	2	34	5.13	0.73	2	66	6.67	0.94
LHR14	[38] light hydrocarbon recovery	14270	321988	22.56	1	64	26.26	1.16	2	37	15.98	0.71
ONETONE1	[38] nonlinear analog circuit	36057	368055	10.21	2	82	14.32	1.40	2	162	17.85	1.75
LHR17	[38] light hydrocarbon recovery	17576	399500	22.73	1	64	26.32	1.16	2	37	15.96	0.70
LHR34	[38] light hydrocarbon recovery	35152	799064	22.73	1	64	26.32	1.16	2	37	15.96	0.70
BCSSTK32	[9] 3D stiffness matrix	44609	1029655	23.08	1	141	10.10	0.44	1	192	10.45	0.45
BCSSTK30	[9] 3D stiffness matrix	28924	1036208	35.83	1	159	21.99	0.61	1	104	15.27	0.43

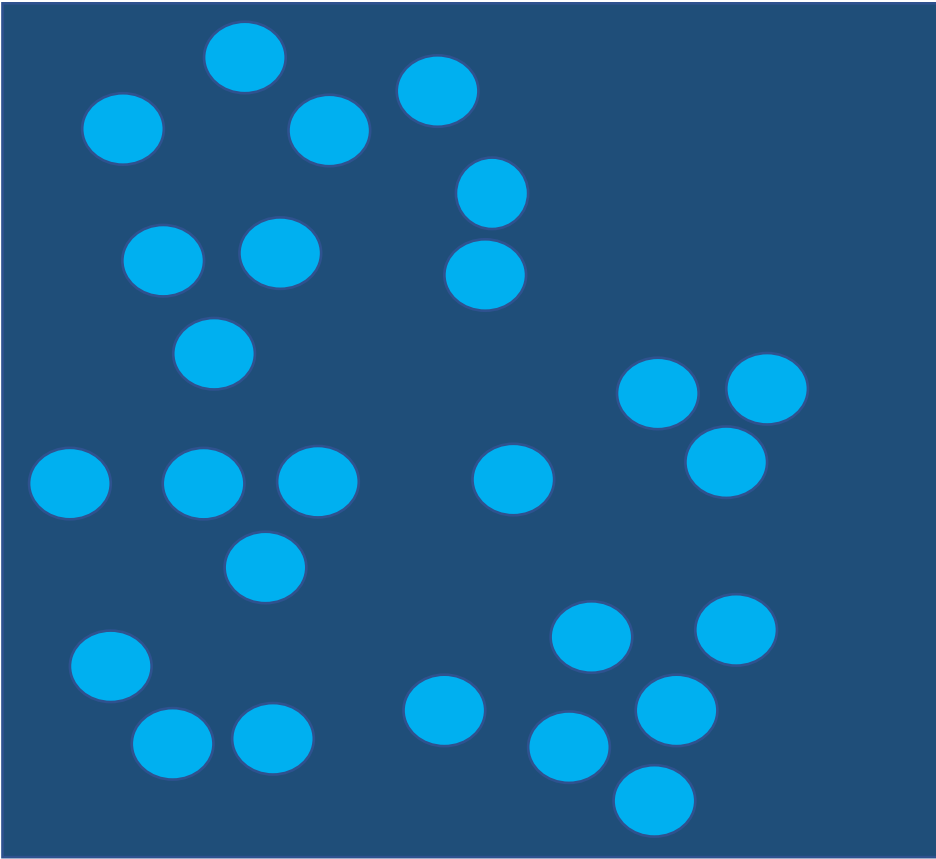
Mathematical Modeling



- Many real-world problems formulated using ordinary and partial differential equations
- Discretization of domain
- Finer the grid, more accurate is the solution

Parallelization of Irregular Application – Cosmology Application

N-body Simulation



Problem

- N bodies exert force on each other
- Model positions of the particles over time

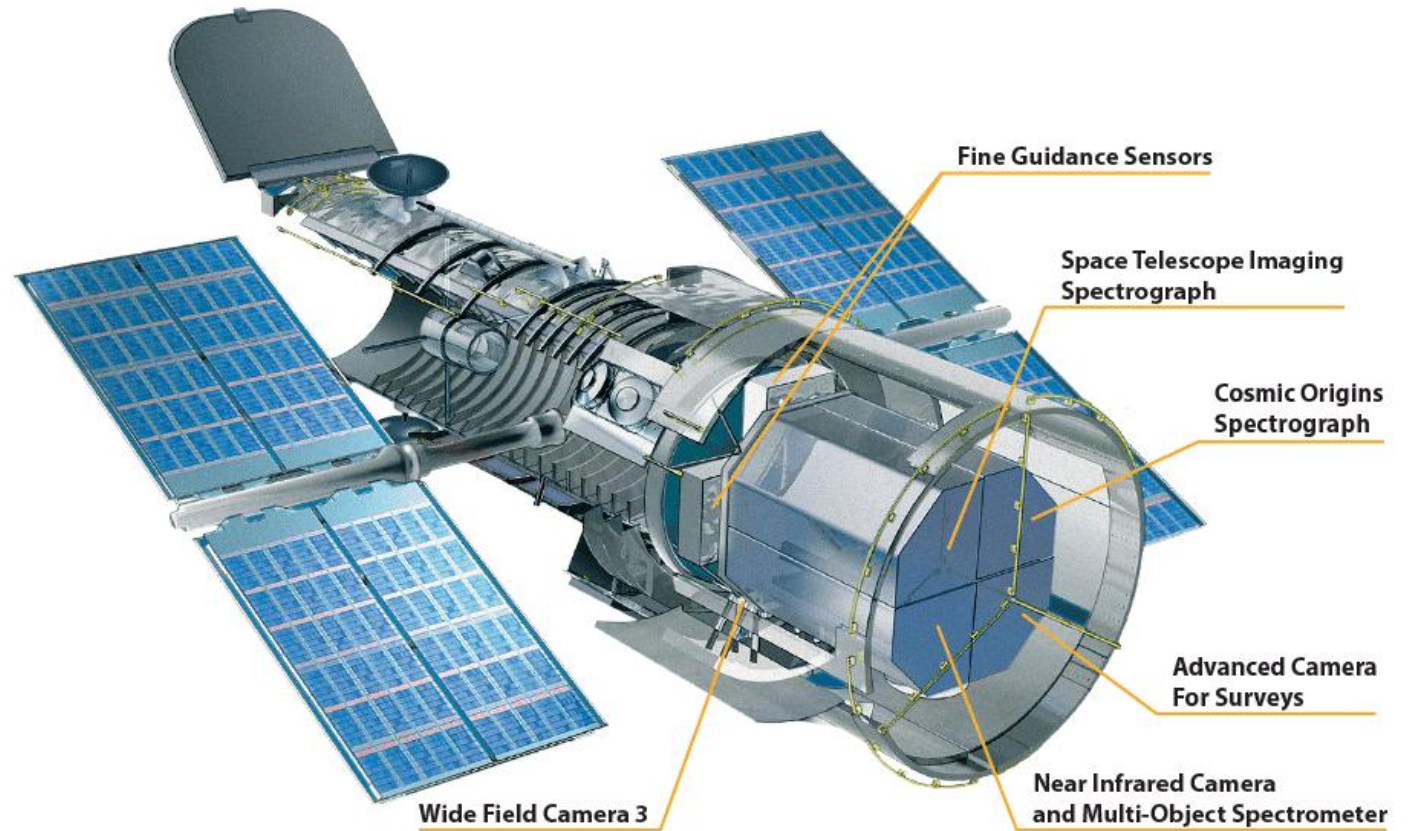
Applications

- Evolution of the universe
- Crack propagation in a material

Sky Observations

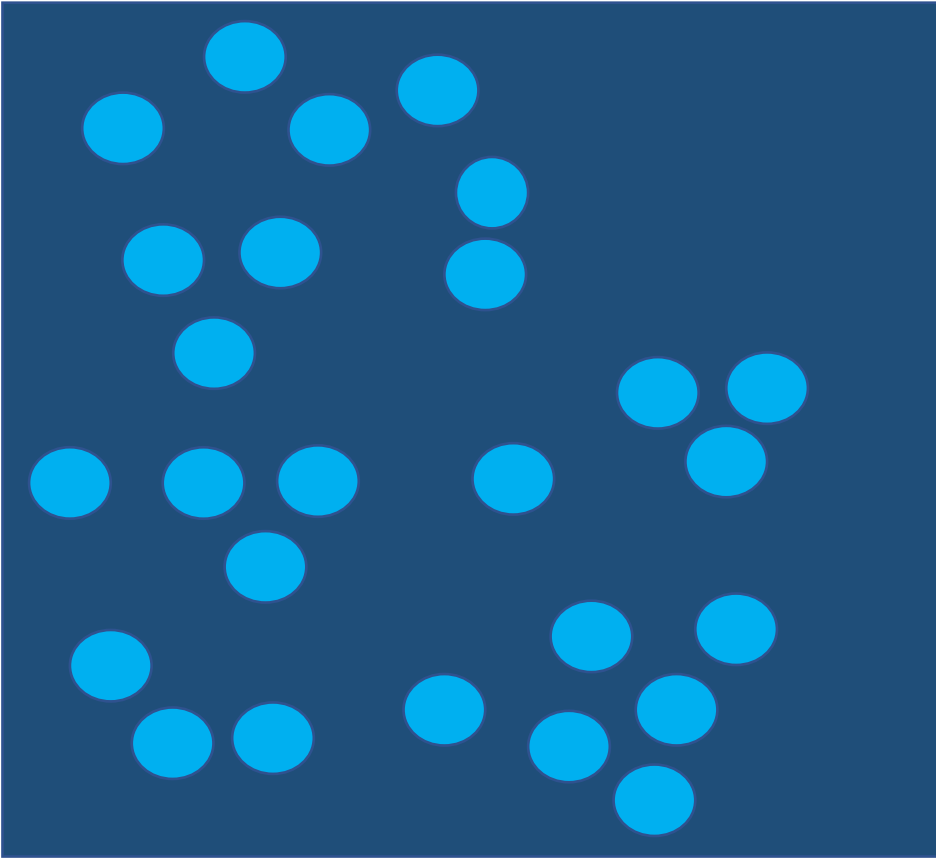


[Source: IDP]



[Source: NASA]

N-body Simulation – Example



Cosmological simulations

- Net force on every celestial object estimated
- Fields – three-dimensional position, velocity, acceleration, and mass
- Thousands of time steps
- Positions updated every time step, evolving system
- Spatial distribution is irregular
- 8000 hrs → 20 hrs [Appel et al., 1985]

Iterative Computations

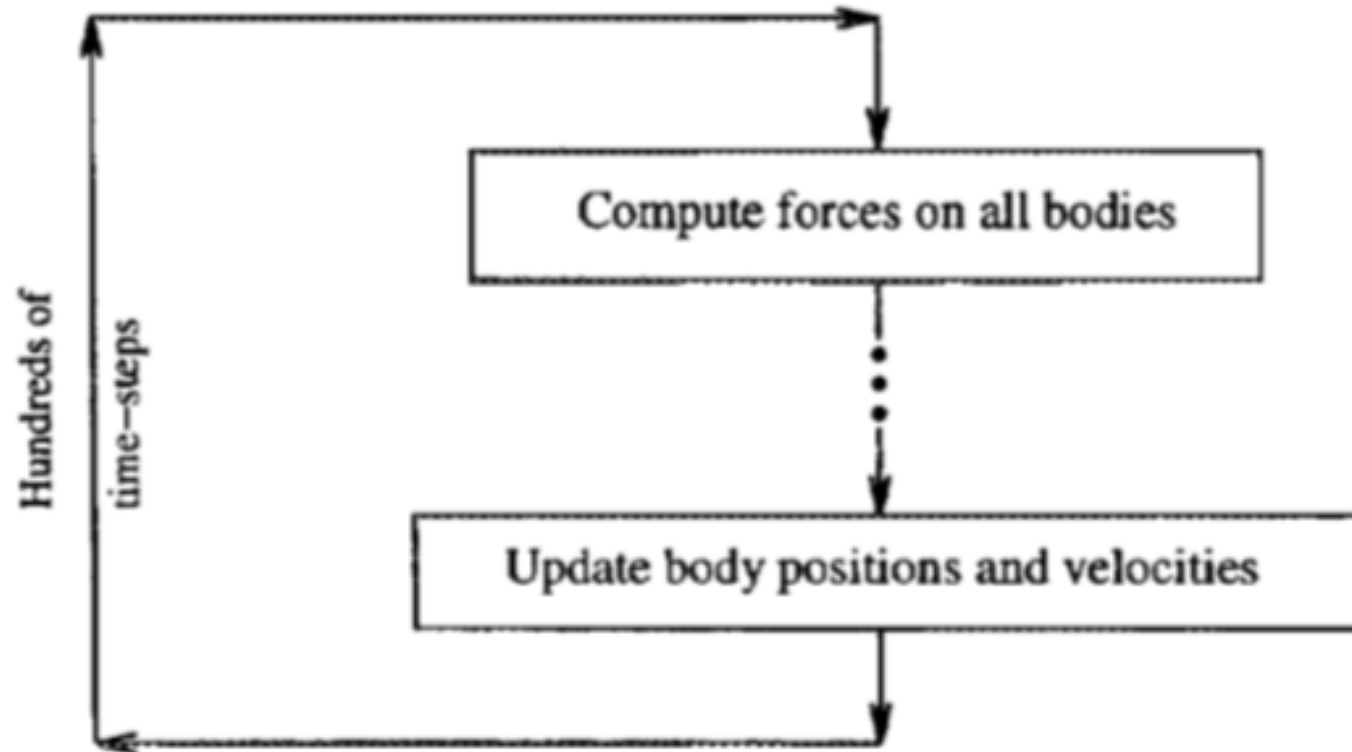
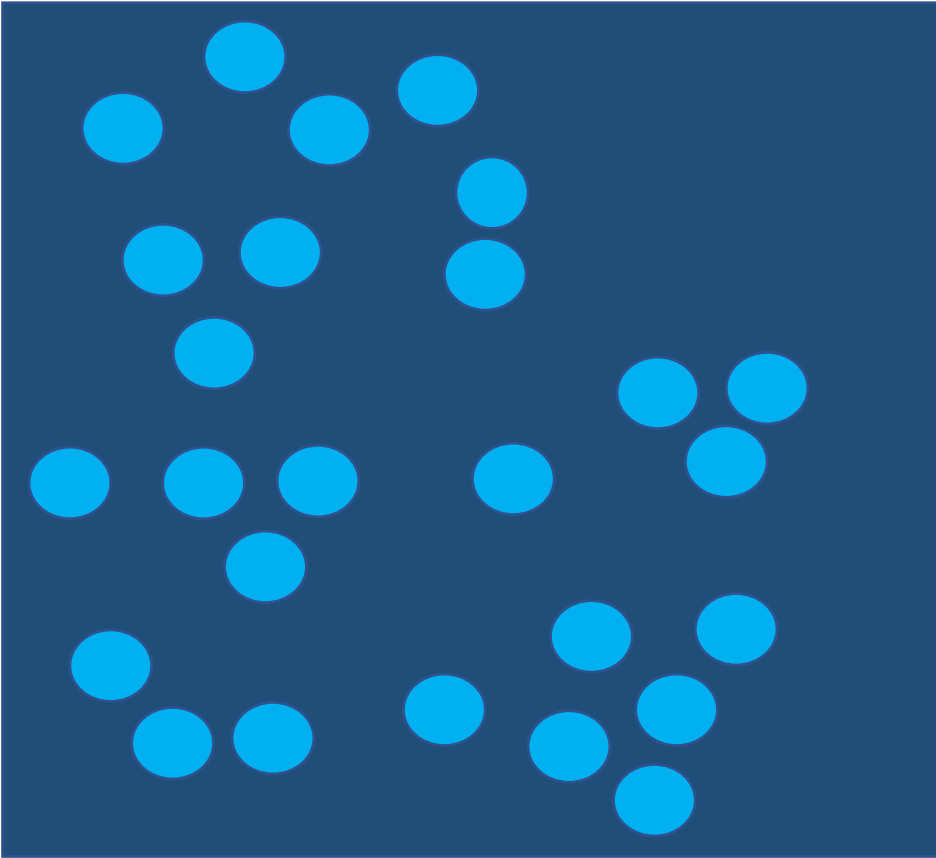


FIG. 1. Top-level flowchart of the classical N -body problem.

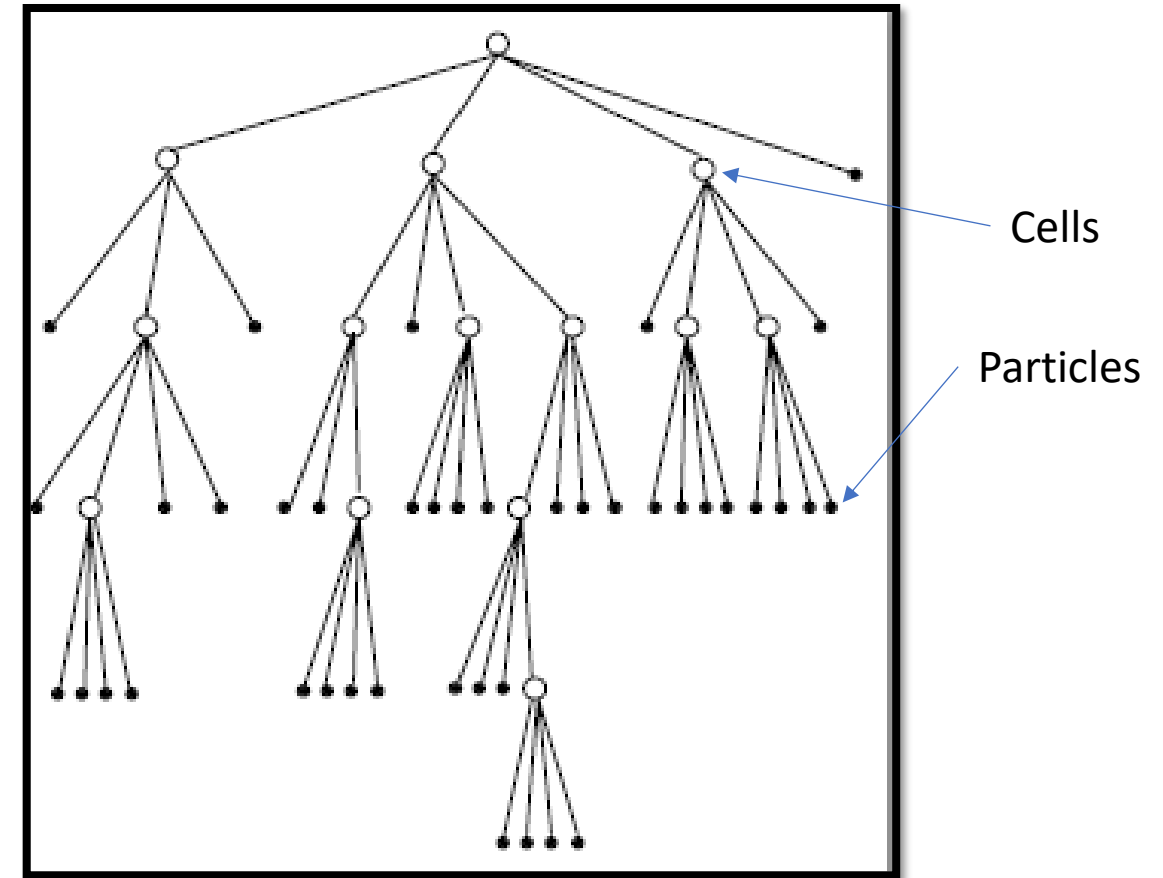
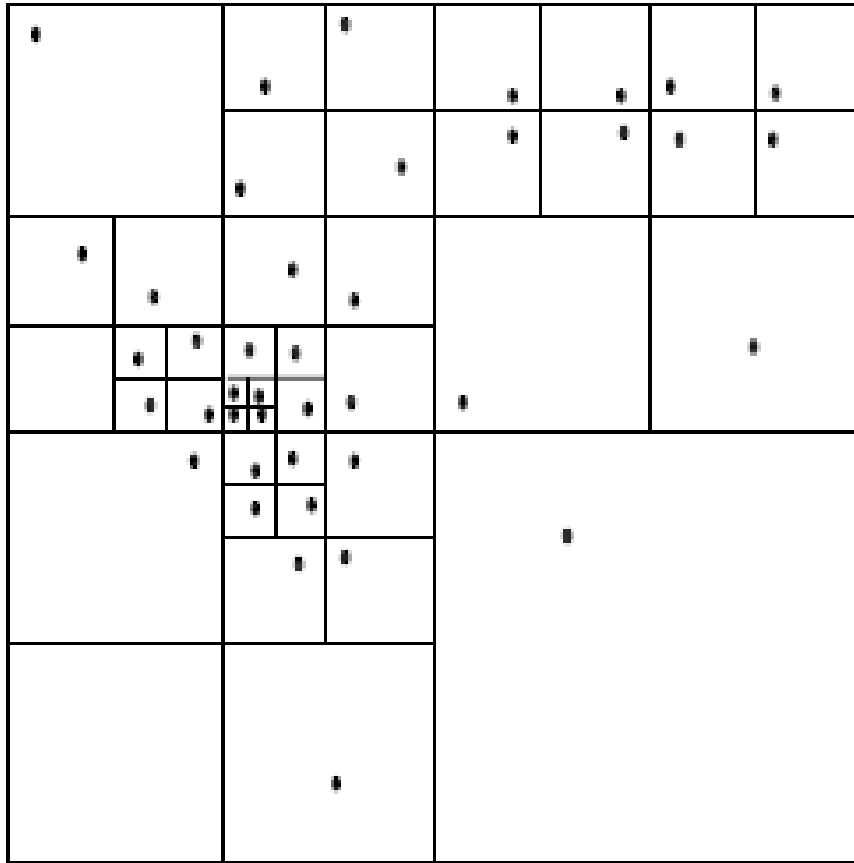
N-body Simulation – Decomposition



[1986] Barnes-Hut method

- Effect of a large group of bodies may be approximated by a single equivalent body
- Center of mass approximation for distant bodies
- Sub-divide the space into 8 sub-regions
- Sub-divide each region based on the number of bodies

Spatial Domain



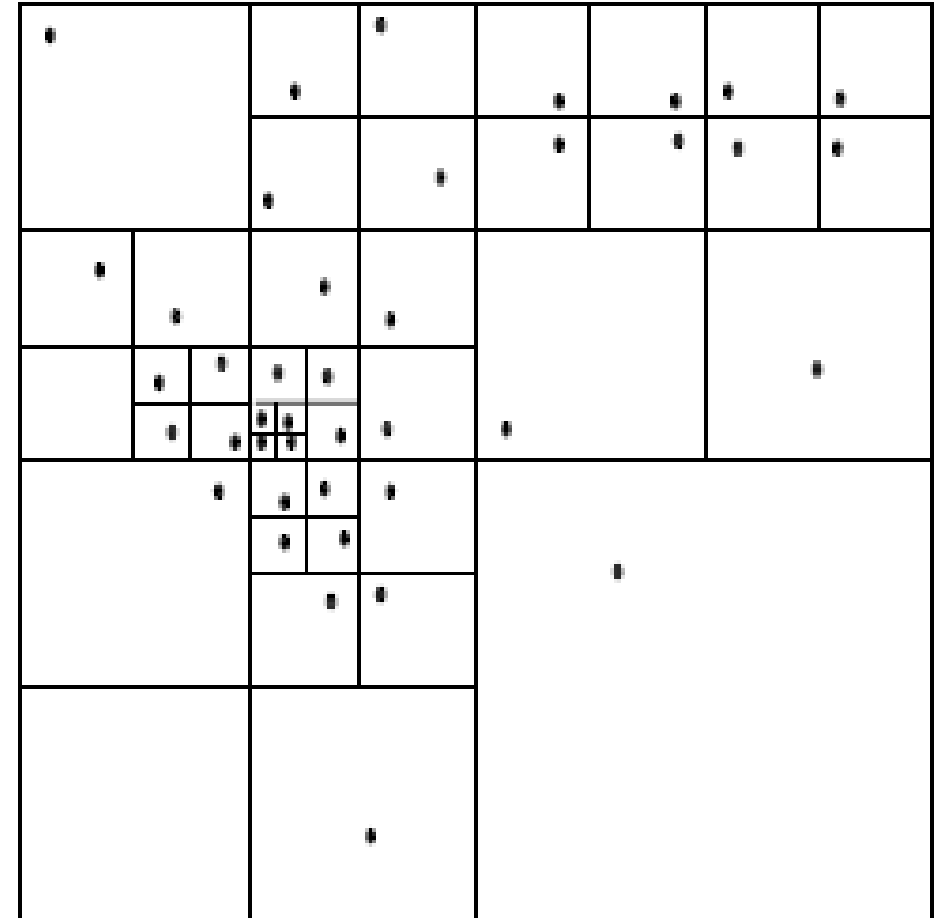
Source: Culler et al. and "Load Balancing and Data Locality in Adaptive Hierarchical N -Body Methods: Barnes-Hut, Fast Multipole, and Radiosity", Singh J.P., Holt C., Totsuka T., Gupta A., Hennessy J.

N-body Simulation

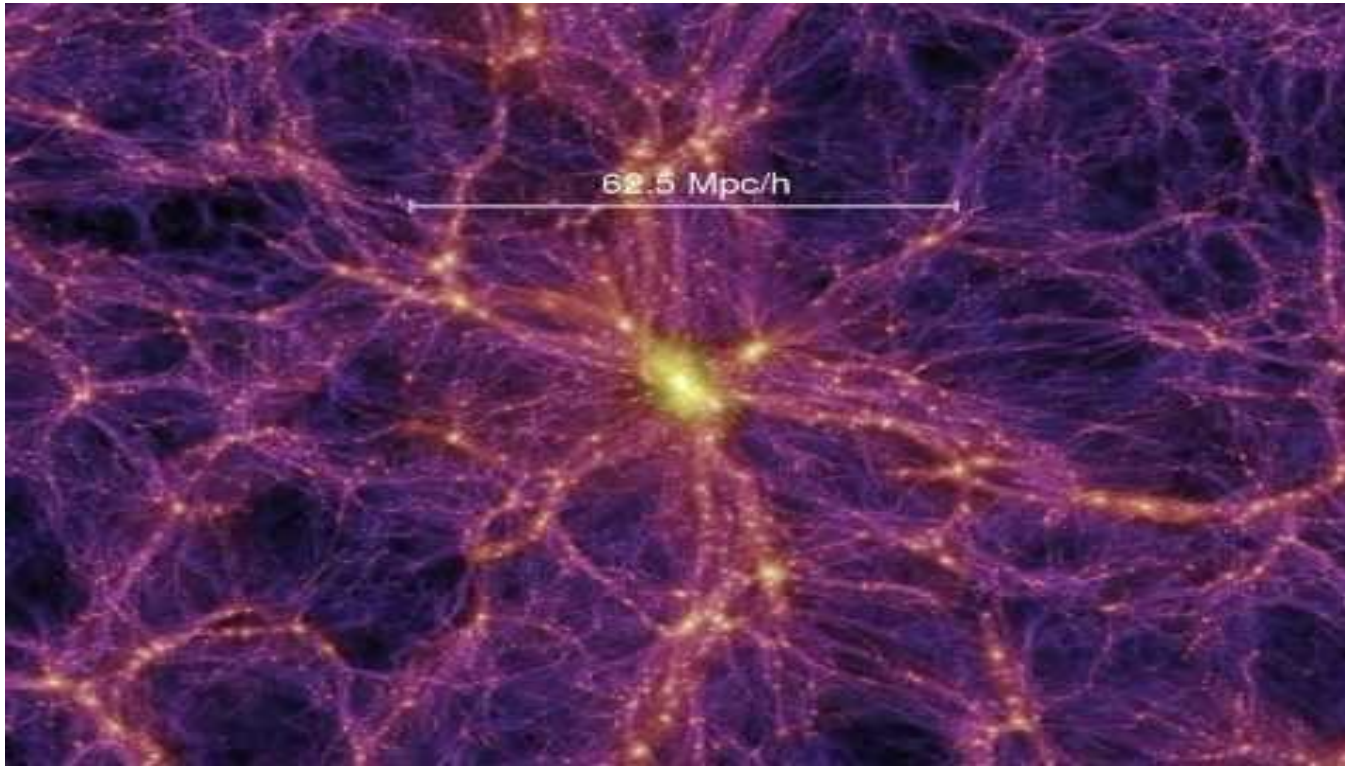
Performance considerations

- Positions change across time steps
 - Dynamic decomposition
 - Repartitioning almost every time step
 - Rebuild the tree every time step
- Irregular communication pattern
 - Logical process neighbors may own spatially dis-contiguous chunks
- Synchronization between steps
- Partitions are not contiguous in space

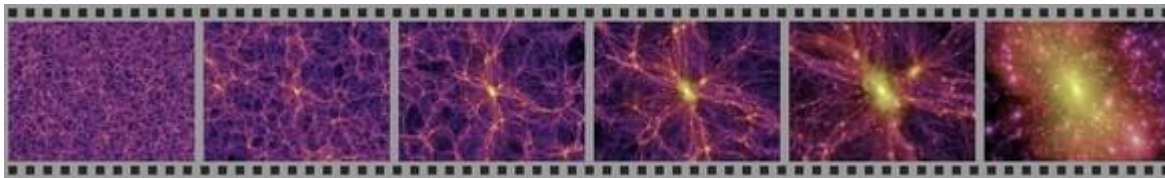
We achieved a good domain decomposition
but ...



The Millennium Simulation (MPA)



"The Millennium Run used more than 10 billion particles to trace the evolution of the matter distribution in a cubic region of the Universe over 2 billion light-years on a side. It kept busy the principal supercomputer at the Max Planck Society's Supercomputing Centre in Garching, Germany for more than a month."



Scalability

“The scalability of a parallel system is a measure of its capacity to increase speedup in proportion to the number of processing elements.” – Introduction to Parallel Computing

Strong scaling

- Fixed problem size
- Increase number of processes
- Efficiency decreases, in general
– why?

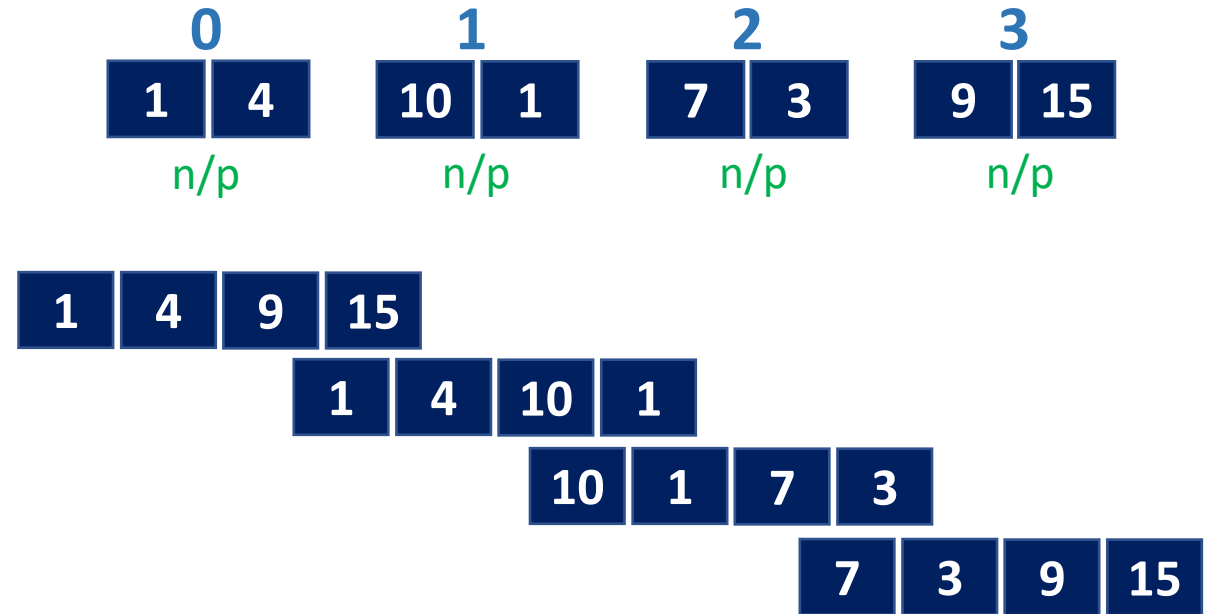
Weak scaling

- Fixed problem size per process
- Increase number of processes
- Increase problem size

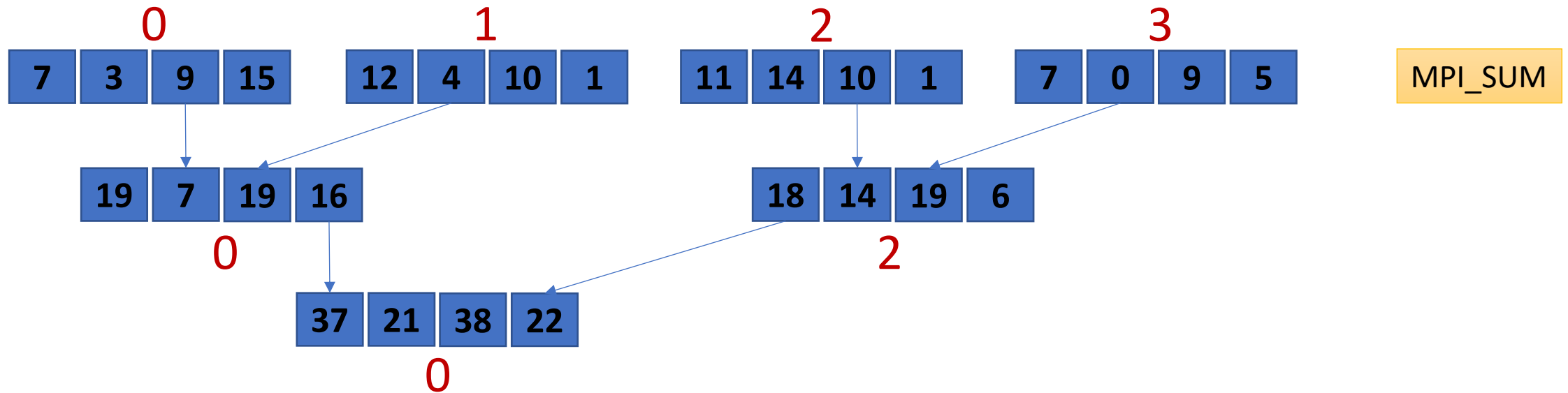
Recap – Algorithms for Collectives

Allgather – Ring Algorithm

- Every process sends to and receives from everyone else
- Assume p processes and total n bytes
- Every process sends and receives n/p bytes
- Time
 - $(p - 1) * (L + n/p * (1/B))$
- How can we improve?



Reduce Algorithm – Recursive doubling

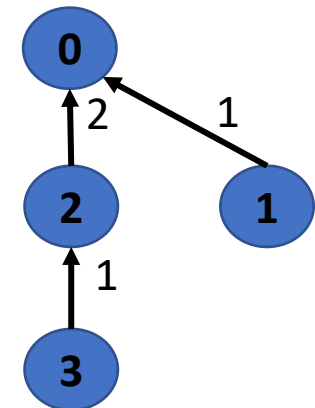


Time: $\log p (L + n \cdot (1/B) + n \cdot c)$

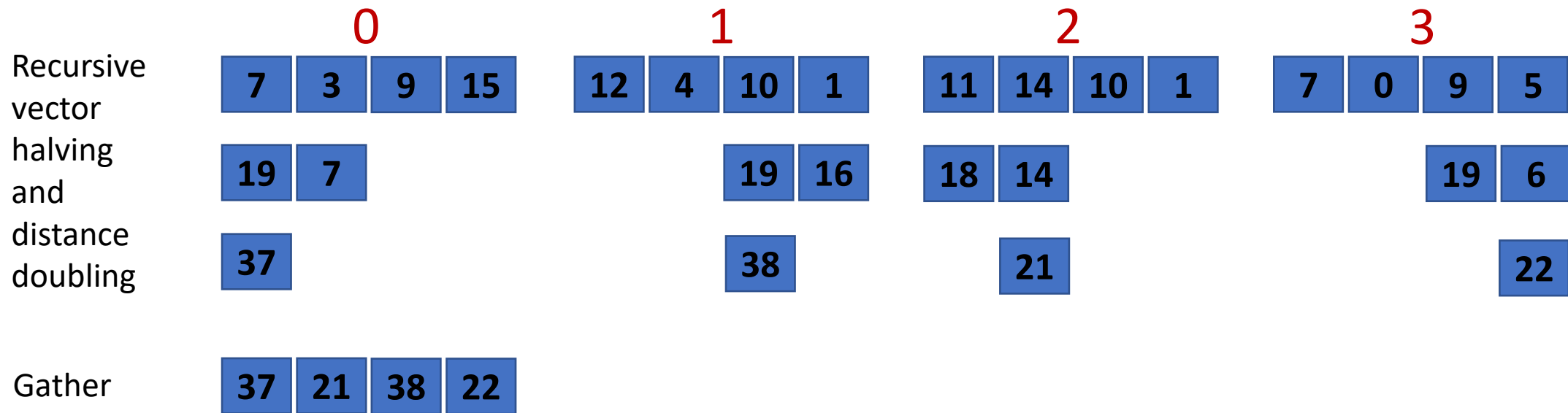
L = latency, B = bandwidth

c = compute cost per byte

Used for short messages



Reduce – Rabenseifner's Algorithm



Time:

$\log p * L + (p-1)/p * (n/B) + (p-1)/p * n * c$ (reduce-scatter) +
 $\log p * L + (p-1)/p * (n/B)$ (gather using binomial)

n = data size

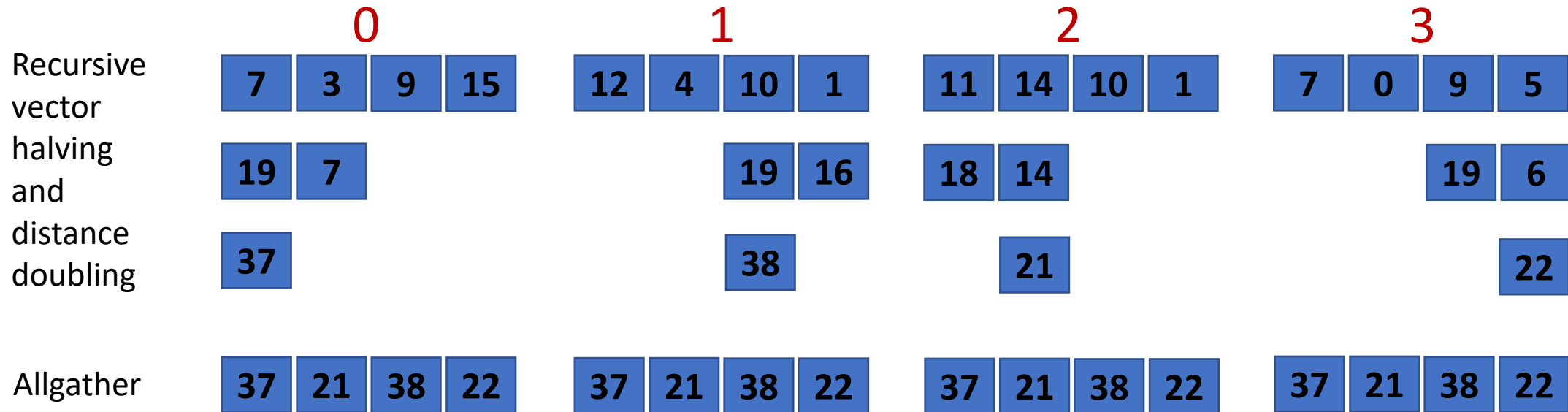
L = latency

p = #processes

B = bandwidth

c = compute cost per byte

Allreduce – Rabenseifner’s Algorithm



Time:

$\log p * L + (p-1)/p * (n/B) + (p-1)/p * n * c$ (reduce-scatter) +

$\log p * L + (p-1)/p * (n/B)$ (allgather using recursive vector doubling and distance halving)

n = data size

L = latency

p = #processes

B = bandwidth

c = compute cost per byte